



Pontificia Universidad Javeriana
Facultad de Ingenieria
Fundamentos de Ingenieria de Software

Informe Final de Entrega

Rubrica, Scrum, QA, DevOps y evidencia tecnica

DriveControl / AutoMinder Enterprise

Equipo SYNTIX TECH

Profesor	Luis Gabriel Moreno Sandoval
Monitor	Juan Pablo Arias Buitrago
Fecha	19 de mayo de 2026
Repositorio	https://github.com/puj-course/FIS_2610_3517_G4

Documento construido con evidencias versionadas, capturas disponibles y trazabilidad verificable. No incluye valores reales de credenciales.

Integrantes

Nombre	Git Hub	Rol Scrum / tecnico
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master
Sebastian Vargas	@juanvargax	Product Owner / Sprint Planner
Samuel Freile	@samuel1680	Configuration Manager
Solon Losada	@solonlosada2006	DevOps Engineer
Sebastian Rodriguez Ramirez	@juserora	QA Lead

Tabla 1: Integrantes y roles del equipo SYNTIX TECH.

Índice

1. Introduccion	3
2. Resumen ejecutivo de cumplimiento de rubrica	3
3. Metodologia agil del semestre	5
3.1. Scrum academico, roles y Definition of Done	5
3.2. Planificacion por sprints y milestones	5
3.3. Metricas agiles del semestre	7
3.4. Metricas agiles detalladas desde Git Hub y Git local	9
3.4.1. Issues por integrante	10
3.4.2. Pull Requests por integrante	11
3.4.3. Commits y lineas de codigo por integrante	12
3.4.4. Milestones y cierre de alcance	13
3.4.5. Reviews por integrante	14
3.5. Trazabilidad agil y cierre tecnico	14
4. Metricas de calidad implementadas en el sistema	14
4.1. Indice de riesgo documental	16
4.2. Completitud de datos operativos	17
4.3. Indice de criticidad de alertas	18
5. SonarCloud	18
5.1. Evidencia complementaria de SonarCloud	21

6. Pruebas unitarias y coverage	23
6.1. Alcance de pruebas	23
6.2. Pruebas frontend	24
6.3. Pruebas backend	25
6.4. Coverage local y SonarCloud	26
7. Docker, despliegue y red	26
7.1. Arquitectura de contenedores	26
7.2. Red Docker	27
8. CI/CD	31
8.1. Workflows	31
9. Integracion SMS / Twilio	35
9.1. Problema inicial y causa	35
9.2. Solucion	35
10.Trazabilidad tecnica	37
11.Postmortem	40
11.1. Que salio bien	40
11.2. Que salio mal y causas raiz	40
12.Conclusiones	42
13.Referencias	42
14.Anexos	42
14.1. Evidencias usadas	43
14.2. Evidencia complementaria de arquitectura	44
14.3. Comandos de validacion	44
14.4. Commit sugerido	45

1. Introduccion

DriveControl / AutoMinder Enterprise es un sistema academico para la gestion de flotas, documentos, alertas operativas y verificacion por mensajeria. La entrega final consolida el trabajo tecnico y metodologico desarrollado por SYNTIX TECH durante el curso Fundamentos de Ingenieria de Software.

El objetivo de este informe es centralizar la evidencia requerida por la rubrica: metricas de calidad, pruebas unitarias, coverage, despliegue reproducible, CI/CD, integracion SMS, trazabilidad Scrum y postmortem. El documento usa evidencia disponible en el repositorio y marca como pendiente cualquier captura o validacion externa que no pueda verificarse desde los archivos versionados.

El alcance de la entrega cubre el informe principal en `docs/Entrega-Final/main.tex`, las evidencias centralizadas en `docs/Entrega-Final/evidencias/` y los anexos de apoyo conservados en `docs/Entrega-Final/anexos/`.

2. Resumen ejecutivo de cumplimiento de rubrica

La siguiente matriz traduce la rubrica oficial a evidencia verificable del repositorio. El estado “Evidenciado” no significa ausencia total de riesgo: significa que existe base tecnica/documental para defender el criterio y que cualquier brecha restante queda declarada con accion concreta.

Criterio	Peso	Exigencia Excelente	Evidencia disponible	Imagen/captura asociada	Estado actual	Riesgo restante	Accion tomada/recomendada
Implementacion de metricas de calidad	20 %	3 metricas propias en codigo, distintas de SonarQube, mas 2 o mas metricas SonarCloud con interpretacion, impacto y acciones.	document-risk, operational-completeness y alert-criticality en apps/web/src/utills/qualityMetrics.js; pruebas en qualityMetrics.test.js; SonarCloud en main.	evidencias/17_metricas_calidad_app.png; evidencias/03_sonarcloud_coverage_main.png.	Evidenciado	Las metricas dependen de los datos de la cuenta activa; una cuenta vacia puede mostrar 0 % o sin datos evaluables.	Se documentan formula, fuente, resultado, interpretacion, impacto y correccion por metrica propia; se separan de Coverage, Duplicidad y Seguridad.
Despliegue y configuracion	20 %	Dockerfile, Docker Compose, build, pruebas, publicacion versionada, despliegue Compose, red Docker y reproduccion documentada.	Dockerfile, apps/web/Dockerfile, docker-compose.yml, docker-compose.prod.yml, make build, make up, healthchecks y red drivecontrol-net.	evidencias/08_make_build.png; evidencias/09_make_up.png; evidencias/11_docker_compose_ps.png; evidencias/12_docker_network_inspect.png.	Evidenciado	DockerHub depende de secrets y de ejecuciones condicionadas; si no se muestra run verde, se debe explicar el alcance.	Se deja procedimiento reproducible, variables solo por nombre, red y limpieza con docker compose down.
Pruebas unitarias	20 %	Coverage mayor a 80 % evidenciado en SonarQube, pruebas criticas y ejecucion automatica en push/PR.	Vitest frontend con LCOV, backend con node -test, workflow sonarcloud.yml; captura SonarCloud de main con 95.6 % coverage.	evidencias/03_sonarcloud_coverage_main.png; evidencias/38_sonarcloud_measures_main.png; evidencias/05_frontend_tests_ok.png; evidencias/07_backend_tests_ok.png.	Evidenciado	Backend se valida con tests Node, pero no esta dentro de sonar.sources; se explica el alcance frontend de SonarCloud.	Se conectan pruebas locales, LCOV, pipeline y SonarCloud; si coverage baja, se priorizan ramas criticas y adaptadores.
Integracion servicio SMS	10 %	Integracion funcional y comprobable con proveedor SMS, con evidencia de envio o recepcion exitosa.	Servicio Twilio en backend/services/smsService.js, OTP en backend, variables por nombre, pruebas mockeadas y capturas academicas controladas.	evidencias/18_sms_app_verificacion.png; evidencias/19_sms_twilio_recibido.png.	Evidenciado	Las capturas pueden contener datos sensibles; no se transcriben valores en texto ni se muestra .env.	Se documenta proveedor, flujo OTP, pruebas, manejo de credenciales y conservacion academica de apps/web/.env.
Metodologias agiles y post-mortem	30 %	Milestones finalizados, HU trazables, HU por sprint, promedio HU/sprint, commits por sprint, participacion, PRs, ramas y postmortem profundo.	docs/Agile/reporte_final_sprints.md: 13 sprints, 88 HU, 375 issues, 405 commits, promedio 6.7 HU/sprint; M1-M4 cerrados; PR #623 y #625 mergeados. Git local: 698 commits en todas las ramas.	Milestones cerrados, tablero Project, contributors, issues #618-#622 y PRs finales.	Evidenciado	Issues/PRs/reviews por integrante requieren export completo con GitHub CLI; alias de Git se normalizan.	Se enfoca el semestre completo y PRs finales quedan como cierre tecnico, no como unico eje agil.

3. Metodologia agil del semestre

La gestion agil se documenta como un proceso de semestre, no como una lista de actividades del Sprint 13. La fuente principal es `docs/Agile/reporte_final_sprints.md`, complementada con los postmortems de `docs/Agile/evidencias_finales/` y con las capturas de milestones e Insights incluidas en este informe.

3.1. Scrum academico, roles y Definition of Done

El equipo trabajo con Scrum academico apoyado por GitHub Issues, milestones, ramas de trabajo, Pull Requests y evidencias por entrega. El flujo defendible fue: planeacion de sprint, division de HU en issues, desarrollo en rama, validacion local, PR, checks automatizados, merge y postmortem.

Integrante	Rol	Responsabilidad defendible
Sebastian Ramirez Maldonado	Scrum Master	Seguimiento de sprints, cierre de evidencia, trazabilidad final y coordinacion de bloqueos.
Sebastian Vargas	Product Owner / Sprint Planner	Priorizacion, planeacion de historias y consolidacion del backlog.
Samuel Freile	Configuration Manager	Configuracion, variables de entorno, consistencia de entorno y soporte de integracion.
Solon Losada	DevOps Engineer	Docker, red, pipelines, publicacion de imagenes y automatizacion de despliegue.
Sebastian Rodriguez Ramirez	QA Lead	Pruebas, coverage, SonarCloud, revision de calidad y evidencias de QA.

Tabla 3: Roles Scrum y responsabilidades principales del semestre.

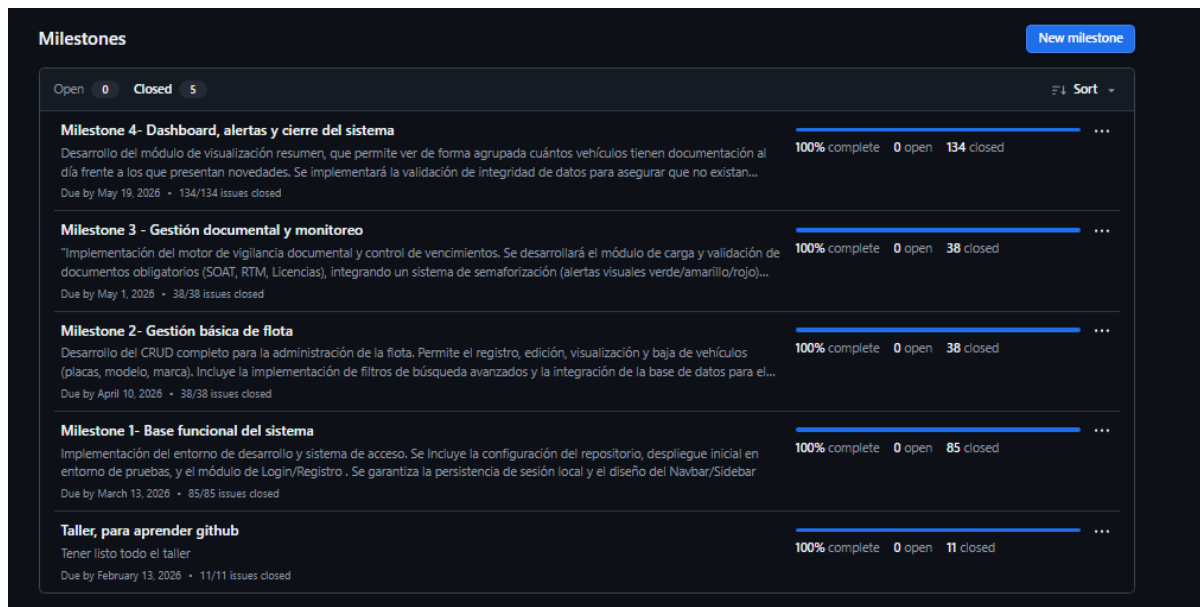
El Definition of Done usado para defender la entrega exige que una HU cierre con codigo integrado, validacion local o pipeline, evidencia funcional cuando aplique, actualizacion documental y trazabilidad issue-commit-PR. Esta regla explica por que las metricas, Docker, SMS y SonarCloud aparecen vinculados a issues y PRs, no solo a capturas aisladas.

3.2. Planificacion por sprints y milestones

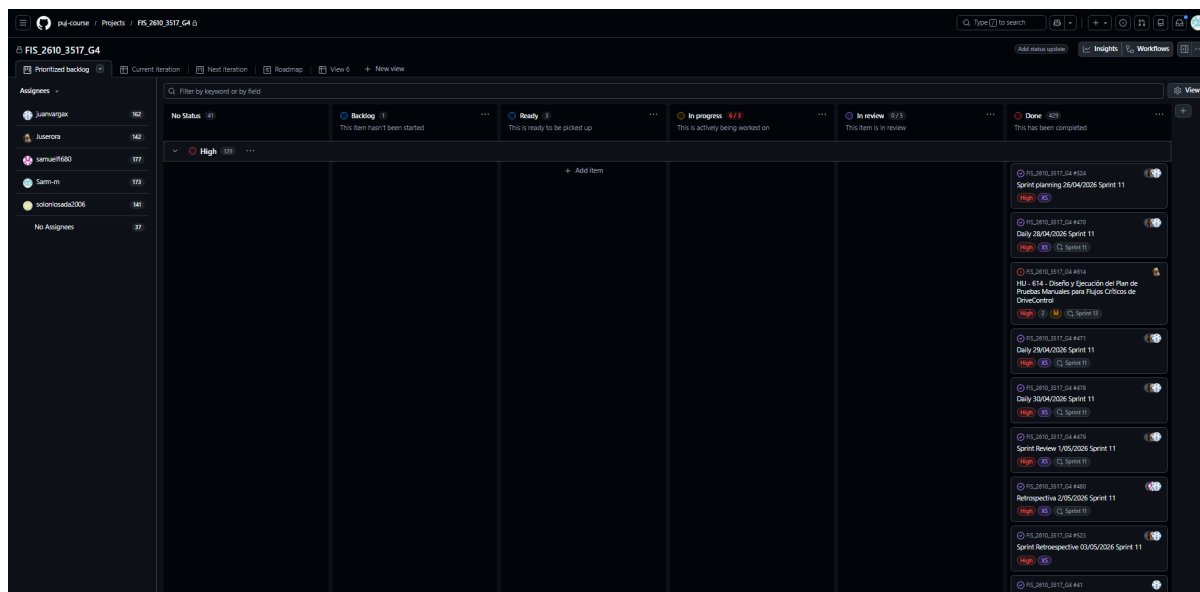
Milestone	Enfoque	Progreso	Estado
M1	Base funcional: repositorio, autenticacion inicial, frontend/backend y convenciones de trabajo.	100 %	Finalizado
M2	Gestion basica de flota, CRUD, rutas, datos y primeros patrones de arquitectura.	100 %	Finalizado
M3	Gestion documental, monitoreo, validaciones, reportes y madurez de QA.	100 %	Finalizado
M4	Dashboard, alertas, SonarCloud, Docker, SMS, CI/CD y cierre de evidencia.	100 %	Finalizado

Tabla 4: Milestones del semestre segun la documentacion agil principal.

El reporte final registra 13 sprints, 88 historias de usuario cerradas, 375 issues y 405 commits, con promedio de 6.7 HU por sprint. Estas cifras permiten evidenciar continuidad de trabajo, capacidad incremental y cierre metodológico; no sustituyen las capturas de GitHub, por lo que se anexan o reservan espacios visuales para tablero, issues y PRs.



Evidencia de milestones cerrados en GitHub.



Tablero GitHub Projects del semestre: columnas de flujo, issues por estado y distribucion por integrantes. Soporta metodologia agil y trazabilidad de trabajo continuo.

3.3. Metricas agiles del semestre

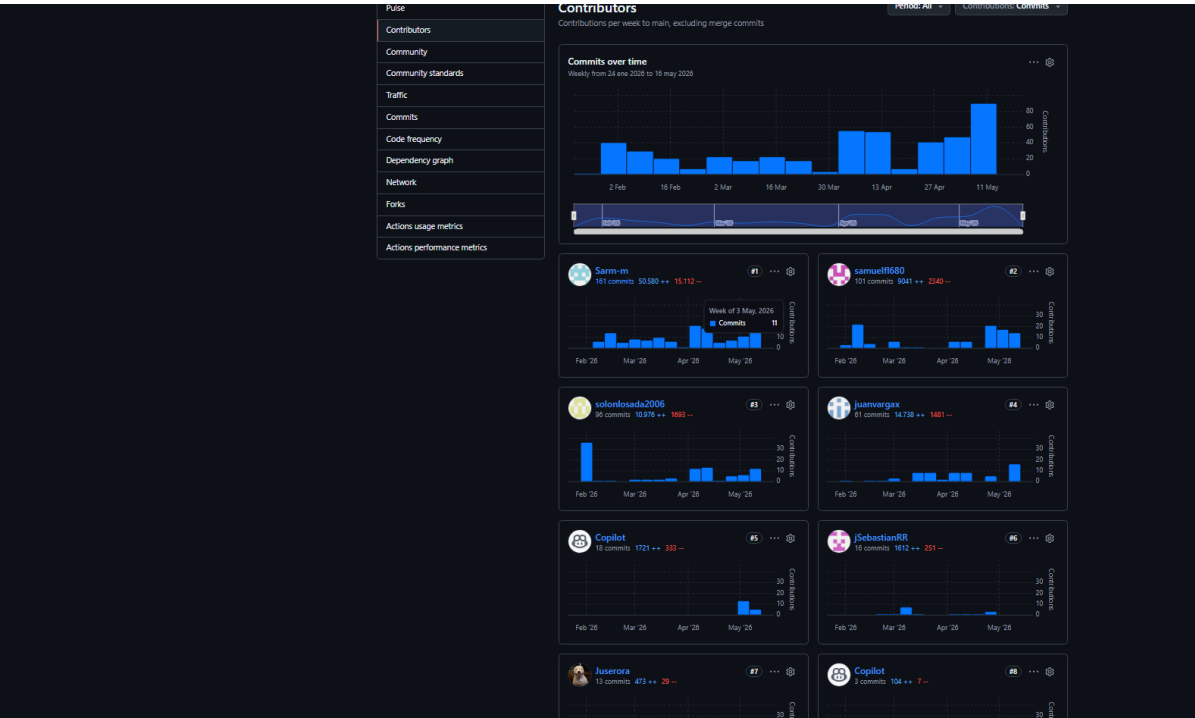
Indicador	Resultado documentado	Interpretacion
Sprints ejecutados	13	Muestra seguimiento regular durante el semestre y no solo cierre final.
HU cerradas	88	Evidencia division del alcance en unidades verificables.
Issues registradas	375	Refuerza trazabilidad fina entre backlog, tareas tecnicas y evidencia.
Commits registrados	405	Indica continuidad tecnica; para participacion individual deben normalizarse alias de Git.
Promedio HU/sprint	6.7	Permite interpretar capacidad historica y balance de carga por iteracion.

Tabla 5: Metricas agiles consolidadas desde el reporte final de sprints.

Sprint	HU cerradas	Issues	Commits	Lectura para la rubrica
Sprint 1	5	44	20	Inicio de base funcional y convenciones del equipo.
Sprint 2	2	12	7	Ajuste de alcance y estabilizacion temprana.
Sprint 3	12	31	20	Incremento de funcionalidades y tareas tecnicas.
Sprint 4	18	57	38	Mayor volumen de HU cerradas del semestre.
Sprint 5	6	33	22	Continuidad de desarrollo con backlog acotado.
Sprint 6	6	30	17	Avance incremental estable.
Sprint 7	4	28	10	Sprint de menor carga funcional documentada.
Sprint 8	5	42	55	Aumento fuerte de commits por integracion tecnica.
Sprint 9	7	31	54	Consolidacion de modulos y QA.
Sprint 10	5	26	15	Cierre de tareas puntuales y ajustes.
Sprint 11	5	23	41	Fortalecimiento tecnico previo al cierre.
Sprint 12	6	10	47	Preparacion de entrega final.
Sprint 13	7	8	59	Cierre tecnico, evidencia, Docker, SonarCloud, SMS y PRs finales.

Tabla 6: HU, issues y commits por sprint segun el reporte final de sprints.

El conteo oficial del reporte agil registra 405 commits asociados al seguimiento del semestre. Como contraste tecnico, git local sobre todas las ramas muestra 698 commits; esta diferencia se explica por merges, ramas remotas, commits de integracion y criterios distintos de corte. Para no inflar artificialmente la metodologia, el informe usa 405 como cifra agil y 698 solo como referencia de auditoria tecnica.



Evidencia de contribucion del equipo en GitHub Insights.

La participacion del equipo debe explicarse con commits, issues, PRs, revisiones, QA, documentacion y responsabilidades de rol. Los conteos por autor requieren cuidado porque Git puede mostrar alias, correos institucionales y correos personales para la misma persona.

Integrante normalizado	Commits	Lineas agregadas	Lineas eliminadas	Interpretacion de participacion
Sarm-m	237	51667	15287	Alta participacion en cierre, documentacion tecnica, integracion y evidencia.
samuelf680	162	9269	2351	Participacion fuerte en configuracion, backend y soporte tecnico.
juanvargax / juansebastianvd	114	15472	1607	Participacion en producto, funcionalidades y planeacion; se agrupan alias asociados al mismo frente de trabajo.
solonlosada2006	109	10850	1567	Participacion visible en DevOps, Docker, pipelines y despliegue.
juserora	51	3393	341	Participacion en QA, pruebas, reportes y validacion.

Tabla 7: Distribucion tecnica por integrante desde git local con alias normalizados.

```

sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ git shortlog -sne --all
~
(END)
128 Sarm-m <ramirezm.sa@javeriana.edu.co>
109 solonlosada2006 <solon.losada2006@gmail.com>
79 samuelfl680 <74272609+samuelfl680@users.noreply.github.com>
74 Sarm <ramirezm.sa@javeriana.edu.co>
64 samuelfl680 <freilesamuel3@gmail.com>
48 juansebastianvd <86634505+juanvargax@users.noreply.github.com>
28 juserora <rodriguezjuans@javeriana.edu.co>
24 juansebastianvd <juansebastianvd@gmail.com>
24 juanvargax <juansebastianvd@gmail.com>
22 Sarm-m <sebastirama@gmail.com>
18 Juan Sebastian Vargas Dicelys <86634505+juanvargax@users.noreply.github.com>
14 Juserora <jsrodriguezz12@gmail.com>
13 Sarm <sebastirama@gmail.com>
12 Samuel felipe <freilesamuel3@gmail.com>
11 Aulas Ingenieria <aulasingenieria@ujaverianas.loc>
7 Juserora <jsrodriguezz12@gmail.com>
6 TuNombre <tuemail@example.com>
6 copilot-swe-agent[bot] <198982749+Copilot@users.noreply.github.com>
3 samuelfl680 <freilsamuel3@gmail.com>
2 Juan Pablo Arias Buitrago <126124282+JuanParias29@users.noreply.github.com>
2 juserora <jsrodriguezz12@gmail.com>
2 unknown <freilesamuel3@gmail.com>
1 Juan Sebastian <freilesamuel3@gmail.com>
1 Samuel Freile <freilesamuel3@gmail.com>
~
~

```

Evidencia visual de commits por integrante. Complementa la lectura Scrum porque muestra participacion tecnica y necesidad de normalizar alias de Git antes de interpretar contribucion individual.

3.4. Metricas agiles detalladas desde GitHub y Git local

Para reforzar el criterio de metodologias agiles y postmortem, se realizo una exportacion adicional desde GitHub CLI y Git local. Esta exportacion complementa el reporte agil del semestre, el cual registra 13 sprints, 88 historias de usuario cerradas, 375 issues de seguimiento, 405 commits y un promedio de 6.7 HU por sprint.

La exportacion completa desde GitHub lista 471 issues en total. Esta cifra no contradice el reporte agil de 375 issues, porque responde a un alcance distinto: incluye issues historicas, todos los estados, labels, asignaciones, elementos sin milestone y registros posteriores usados para cierre tecnico. Para no inflar artificialmente el avance agil, el informe conserva las 375 issues como cifra oficial del reporte de sprints y usa las 471 issues como auditoria completa del repositorio.

Los datos finos se obtuvieron desde los anexos:

- docs/Entrega-Final/anexos/issues.json
- docs/Entrega-Final/anexos/pull_requests.json

- docs/Entrega-Final/anexos/contributors.json
- docs/Entrega-Final/anexos/commits_por_autor.txt
- docs/Entrega-Final/anexos/commits_detalle.tsv
- docs/Entrega-Final/anexos/lineas_por_autor_raw.txt
- docs/Entrega-Final/anexos/lineas_por_autor_resumen.tsv

Metrica exportada	Resultado	Interpretacion para la rubrica
Issues totales exportadas desde GitHub	471	Permite auditar el backlog completo del repositorio, incluyendo historico, issues abiertas, cerradas, asignadas y sin milestone.
Issues cerradas	460	Representa un cierre global de 97.7 %, evidencia fuerte de seguimiento y gestion del trabajo.
Issues abiertas	11	Refleja pendientes menores o elementos residuales que no bloquean el cierre tecnico, pero deben mencionarse como backlog restante.
Pull Requests totales	159	Evidencia uso constante de integracion por PR y control de cambios durante el semestre.
Pull Requests mergeadas	133	Representa 83.6 % de PRs integradas, lo que soporta control de versiones y trazabilidad de entregas.
Pull Requests cerradas sin merge	25	Indica trabajo descartado, reemplazado o no integrado; es normal en un proceso iterativo si se justifica en postmortem.
Pull Requests abiertas	1	Debe revisarse como pendiente operativo si sigue abierta al cierre.
Commits en Git local, todas las ramas	698	Cifra de auditoria tecnica que incluye ramas, merges e integraciones; complementa los 405 commits del reporte agil oficial.
Commits del equipo principal	673	Representa 96.4 % de los commits locales, evidenciando participacion directa del equipo.

Tabla 8: Resumen de metricas finas exportadas desde GitHub y Git local.

3.4.1. Issues por integrante

Las issues pueden tener mas de un asignado. Por esa razon, la columna de asignaciones no suma 471, sino que representa participacion acumulada por responsabilidad asignada. Esta lectura es mas adecuada para Scrum porque permite observar carga de trabajo y seguimiento por integrante, no solo quien creo la issue.

Integrante	Issues creadas	Asignaciones totales	Asignaciones cerradas	Lectura agil
Sarm-m	147	170	164	Alta participacion en seguimiento, cierre de evidencia, Scrum y coordinacion tecnica.
juanvargax	118	161	155	Participacion alta en producto, backlog, planeacion y seguimiento de historias.
samuelfl680	88	177	170	Mayor numero de asignaciones; evidencia fuerte de carga tecnica y soporte de configuracion.
solonlosada2006	77	149	143	Participacion relevante en DevOps, Docker, pipelines y tareas de despliegue.
juserora	41	136	126	Participacion en QA, pruebas, validaciones y cierre de calidad.
Sin asignar	—	35	35	Issues cerradas sin responsable explicito; deben interpretarse como registros de seguimiento general.

Tabla 9: Issues creadas y asignaciones por integrante a partir de `issues.json`.

3.4.2. Pull Requests por integrante

Los Pull Requests permiten evidenciar control de versiones, integracion incremental y revision del trabajo. En esta tabla se contabiliza el autor del PR y su estado final.

Integrante	PRs totales	PRs mergeadas	PRs cerradas sin merge	Interpretacion
Sarm-m	67	54	13	Mayor participacion en integracion y cierre tecnico; incluye PRs de consolidacion y evidencia.
samuelfl680	35	28	7	Participacion fuerte en integracion tecnica y configuracion.
juanvargax	23	21	2	Alta tasa de integracion, relacionada con producto y avance funcional.
solonlosada2006	18	16	2	Participacion asociada a DevOps, despliegue y soporte tecnico.
juserora	16	14	1 cerrada y 1 abierta	Participacion en QA y validaciones; queda una PR abierta que debe revisarse si persiste al cierre.

Tabla 10: Pull Requests por integrante desde `pull_requests.json`.

3.4.3. Commits y lineas de codigo por integrante

Git puede mostrar distintos alias, correos personales, correos institucionales o cuentas automaticas. Por eso, los datos se presentan con normalizacion de alias. La lectura no debe usarse como unico indicador de trabajo individual, sino como evidencia complementaria junto con issues, PRs, revisiones, pruebas, documentacion y rol Scrum.

Integrante normalizado	Commits	Lineas agregadas	Lineas eliminadas	Interpretacion de participacion
Sarm-m	237	51667	15287	Alta participacion en cierre, documentacion tecnica, integracion y evidencia.
samuelfl680	162	9269	2351	Participacion fuerte en configuracion, backend y soporte tecnico.
juanvargax / juansebastianvd	114	15472	1607	Participacion visible en producto, funcionalidades y planeacion; se agrupan alias del mismo frente de trabajo.
solonlosada2006	109	10850	1567	Participacion visible en DevOps, Docker, pipelines y despliegue.
juserora	51	3393	341	Participacion en QA, pruebas, reportes y validacion.

Tabla 11: Commits y lineas por integrante desde Git local con alias normalizados.

3.4.4. Milestones y cierre de alcance

El archivo `milestones.json` puede quedar vacío si la consulta de GitHub CLI no trae milestones cerrados; sin embargo, cada issue exportada conserva la información de milestone asociada. Por eso se consolida el avance de milestones a partir de `issues.json`.

Milestone	Issues totales	Issues cerradas	Issues abiertas
Milestone 4- Dashboard, alertas y cierre del sistema	132	132	0
Milestone 1- Base funcional del sistema	76	76	0
Milestone 3 - Gestion documental y monitoreo	38	38	0
Milestone 2- Gestion basica de flota	37	37	0
Taller, para aprender github	11	11	0
Sin milestone	177	166	11

Tabla 12: Distribucion de issues por milestone a partir de `issues.json`.

La lectura principal para la rubrica es que los milestones funcionales M1–M4 aparecen cerrados en su totalidad dentro de la exportacion de issues. Las 11 issues abiertas se concentran en elementos sin milestone, por lo que no afectan directamente el cierre de los hitos principales, aunque deben mantenerse como backlog residual.

3.4.5. Reviews por integrante

El archivo `pull_requests.json` exportado con GitHub CLI no incluye revisiones por PR. Por esta razón, el informe no inventa reviews por integrante. Para obtenerlas se requiere una exportación adicional usando GraphQL o consultando el endpoint de reviews por cada Pull Request. Mientras no exista esa exportación, la evidencia de participación se sustenta con issues, PRs, commits, líneas, roles Scrum y capturas de GitHub Insights.

Pendiente: Exportar reviews por integrante con GitHub API o GraphQL si el profesor exige ese desglose. No completar manualmente sin fuente verificable.

3.5. Trazabilidad agil y cierre tecnico

Los PRs #623 y #625 corresponden al cierre técnico final: #623 integro `feature-sarm-m` hacia `develop`, y #625 integro `develop` hacia `main`. En metodología ágil se usan como evidencia de integración y cierre, pero no reemplazan la evidencia del semestre completo.

12	Consolidar entrega técnica final según rúbrica #618	Sarm-m	Done	High	M
13	Consolidar métricas de calidad, SonarCloud y coverage #619	Sarm-m	Done	High	M
14	Validar Docker Compose, red Docker y despliegue local #620	Sarm-m	Done	High	M
15	Documentar CI/CD, DockerHub y validación de workflows #621	Sarm-m	Done	Medium	M
16	Consolidar SMS, seguridad de entorno y paquete final de evidencia #622	Sarm-m	Done	High	M

Issues #618 a #622 cerradas y trazables en el cierre técnico.

PR #623 y PR #625 como flujo `feature-sarm-m` -> `develop` -> `main`.

4. Métricas de calidad implementadas en el sistema

Las métricas propias están implementadas como reglas de dominio en `apps/web/src/utills/qualityMetrics.js`. Son distintas de SonarCloud porque miden calidad funcional y riesgo

operativo de la flota: documentos vencidos, completitud de registros y criticidad de alertas. SonarCloud analiza código fuente; no conoce si un SOAT está vencido, si un conductor carece de teléfono o si una alerta roja está bloqueando la operación.

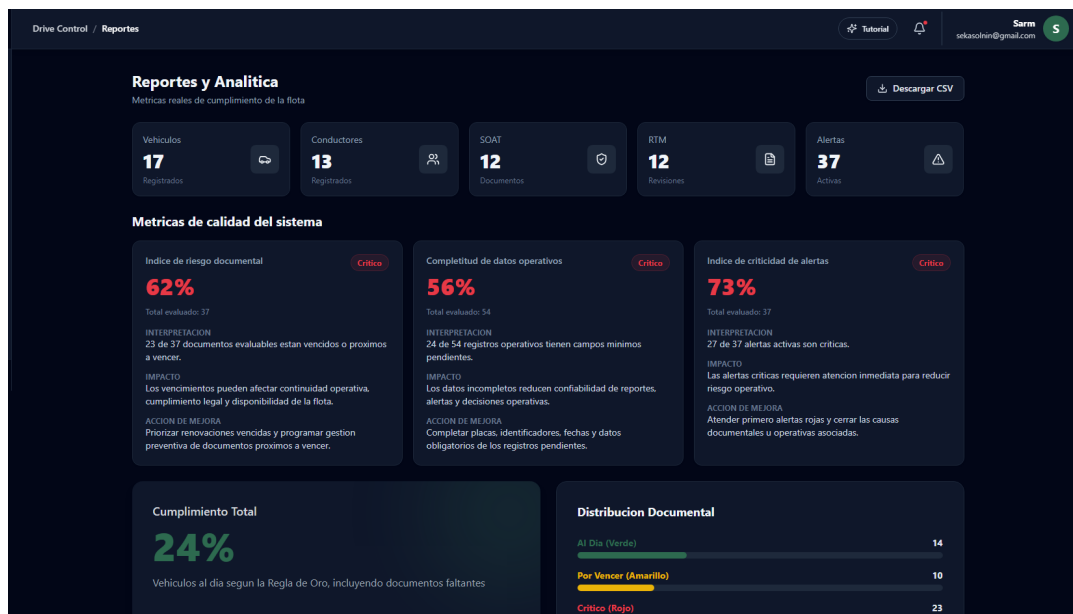


Figura: Métricas de calidad del sistema en la app: índice de riesgo documental, completitud de datos operativos e índice de criticidad de alertas.

La captura muestra los resultados observados de las tres métricas propias en la cuenta usada para la validación. Estas métricas son dinámicas: se calculan con los vehículos, documentos, conductores y alertas registrados por el usuario activo. Por tanto, si se crea una cuenta nueva sin información operativa, los indicadores pueden aparecer en 0% o como “sin datos evaluables”. Eso no invalida la métrica; confirma que el cálculo responde al estado real de la flota registrada. Para reproducir la evidencia se debe usar una cuenta con datos de prueba cargados o registrar vehículos, documentos, conductores y alertas antes de validar.

4.1. Índice de riesgo documental

Campo	Detalle tecnico
Tipo	Metrica propia del sistema, no SonarCloud.
Que mide	Proporcion de documentos evaluables vencidos o proximos a vencer.
Fuente de datos	SOAT, RTM, licencias, documentos genericos y conductores con estado, fecha de vencimiento o dias restantes.
Formula/logica	$(\text{documentos vencidos} + \text{documentos proximos}) / \text{documentos evaluables} * 100$. En codigo, estados rojo y amarillo cuentan como afectados.
Resultado observado	62 %; 23 de 37 documentos evaluables estan vencidos o proximos a vencer; estado critico.
Interpretacion tecnica	Un valor superior al umbral rojo de 50 % indica concentracion alta de riesgo documental y requiere priorizacion inmediata.
Impacto en DriveControl	Afecta cumplimiento legal, continuidad operativa, disponibilidad de vehiculos y confiabilidad de alertas preventivas.
Impacto operativo en flota	La flota puede quedar inmovilizada, recibir sanciones o perder disponibilidad si se opera con documentos vencidos.
Por que SonarCloud no la mide	SonarCloud no evalua vigencia legal ni datos de negocio; solo inspecciona codigo, duplicidad, seguridad, bugs y coverage.
Accion de mejora	Renovar primero documentos vencidos, programar renovaciones proximas y crear seguimiento preventivo por fecha.
Correccion si es critico	Bloquear o marcar vehiculos con documentos rojos, asignar responsable de renovacion y reejecutar la metrica hasta reducir el porcentaje bajo 50 %.

Tabla 13: Interpretacion del indice de riesgo documental.

4.2. Completitud de datos operativos

Campo	Detalle tecnico
Tipo	Metrica propia del sistema, no SonarCloud.
Que mide	Porcentaje de registros de vehiculos, conductores y documentos que cumplen campos minimos obligatorios.
Fuente de datos	Vehiculos, conductores, SOAT, RTM y documentos genericos capturados por el usuario.
Formula/logica	$\text{registros completos} / \text{registros evaluados} * 100$. El codigo evalua grupos de campos requeridos por tipo de registro.
Resultado observado	56 %; 24 de 54 registros operativos tienen campos minimos pendientes; estado critico.
Interpretacion tecnica	Un valor menor a 80 % esta en rojo porque reduce confiabilidad de reportes, alertas y decisiones.
Impacto en DriveControl	Deteriora la trazabilidad y puede generar alertas incompletas o reportes con datos faltantes.
Impacto operativo en flota	Un registro sin placa, identificador, telefono o fecha puede impedir contactar conductores, validar documentos o priorizar mantenimientos.
Por que SonarCloud no la mide	SonarCloud no valida completitud semantica de registros de negocio ni calidad de datos operativos.
Accion de mejora	Completar placas, identificadores, telefonos, fechas, polizas, certificados y campos obligatorios pendientes.
Correccion si es critico	Agregar validaciones de formulario y tareas de depuracion de datos hasta subir la completitud al menos a 80 %, idealmente 100 %.

Tabla 14: Interpretacion de la completitud de datos operativos.

4.3. Índice de criticidad de alertas

Campo	Detalle tecnico
Tipo	Metrica propia del sistema, no SonarCloud.
Que mide	Porcentaje de alertas criticas dentro del total de alertas activas.
Fuente de datos	Alertas generadas por adaptadores de SOAT, RTM, licencias, vehiculos y reglas operativas.
Formula/logica	$\text{alertas criticas} / \text{alertas activas} * 100$. El codigo normaliza estados como rojo, critico, vencido o danger.
Resultado observado	73 %; 27 de 37 alertas activas son criticas; estado critico.
Interpretacion tecnica	Un valor de 30 % o mas es rojo; 73 % significa que la mayoria de alertas requiere accion inmediata.
Impacto en DriveControl	Obliga a priorizar alertas rojas para proteger continuidad, cumplimiento y seguridad operativa.
Impacto operativo en flota	Una alta concentracion de alertas criticas puede indicar documentos vencidos, fallas de mantenimiento o datos incompletos que afectan disponibilidad.
Por que SonarCloud no la mide	SonarCloud no interpreta severidad operacional ni prioriza eventos activos de negocio.
Accion de mejora	Atender primero alertas rojas, cerrar causas documentales u operativas y registrar causa raiz.
Correccion si es critico	Crear cola de atencion por severidad, resolver alertas bloqueantes y medir nuevamente hasta bajar de 30 %.

Tabla 15: Interpretacion del indice de criticidad de alertas.

Evidencia de implementacion:

- Codigo: `apps/web/src/utils/qualityMetrics.js`
- Tests: `apps/web/src/__tests__/qualityMetrics.test.js`
- Generador reproducible: `apps/web/tools/generate-quality-evidence.mjs`
- Comando: `npm -prefix apps/web run quality:metrics`

5. SonarCloud

SonarCloud complementa las metricas de dominio con analisis estatico. La configuracion revisada usa `sonar.organization=puj-course`, `sonar.projectKey=puj-course_FIS_2610_3517_G4`, fuentes en `apps/web/src` y coverage importado desde `apps/web/coverage/lcov.info`.

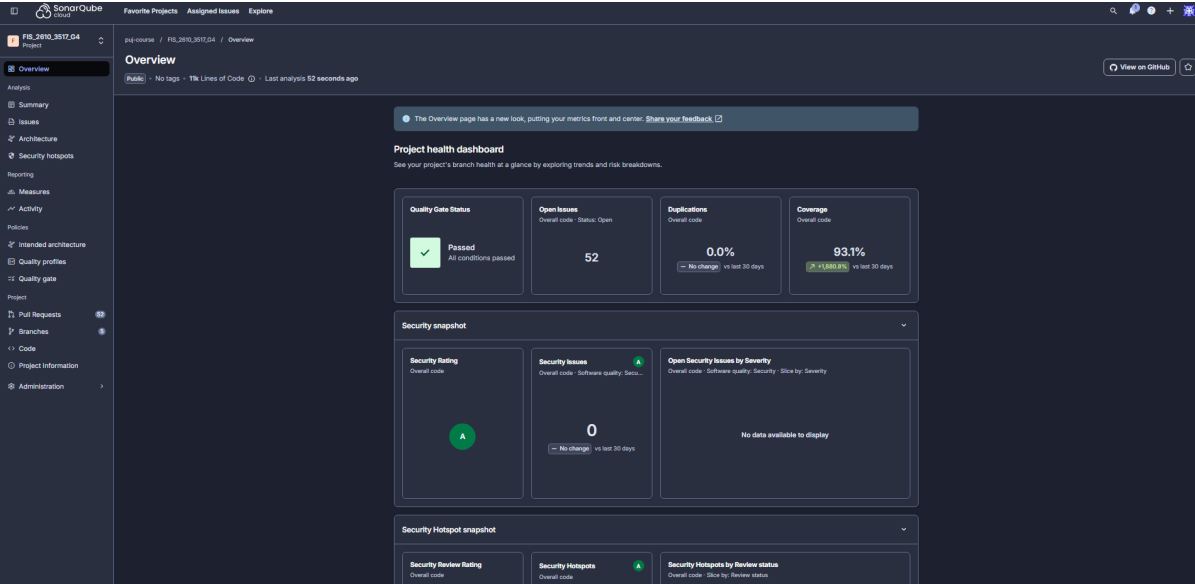
El plan actual de SonarCloud analiza oficialmente la rama `main`. Por eso, `develop` y `feature-sarm-m` pueden aparecer como ramas no analizadas o con visibilidad limitada, mientras que la evidencia oficial se toma desde `main`. El flujo usado fue `feature-sarm-m -> develop -> main`. Los valores reportados a continuacion corresponden a las capturas finales disponibles de Summary/Measures en `main`; no se inventan metricas que no esten respaldadas por evidencia.

Metrica	Que mide	Resultado observado	Interpretacion y accion de mejora
Quality Gate	Cumplimiento global de condiciones de calidad.	Passed.	Todas las condiciones configuradas pasan; si falla, no se debe integrar hasta corregir la condicion bloqueante.
Coverage	Porcentaje de codigo medido ejecutado por pruebas.	95.6 %.	Supera ampliamente el umbral de excelencia de 80 %; si baja, agregar pruebas en ramas no cubiertas y revisar <code>lcov.info</code> .
Duplications	Bloques duplicados detectados por analisis CPD.	0.0 %.	Duplicidad nula o muy baja reduce deuda y bugs repetidos; si sube, extraer helpers o componentes compartidos.
Security Rating	Riesgo de vulnerabilidades detectadas por reglas de seguridad.	A.	Rating A indica estado saludable visible; si baja, corregir vulnerabilidades antes de merge.
Security Issues	Issues de seguridad abiertos.	0.	Cero issues abiertos reduce exposicion; si aparecen, revisar sanitizacion, autenticacion y dependencias.
Security Hotspots	Puntos que requieren revision manual de seguridad.	0; Security Review Rating A.	No hay hotspots abiertos visibles en la captura complementaria; si aparecen en futuras iteraciones, deben revisarse y documentar la decision.
Reliability	Bugs potenciales o riesgos de confiabilidad detectados por analisis estatico.	Reliability Rating A; 19 reliability issues de severidad baja.	La calificacion global A no bloquea el Quality Gate. Como mejora, se deben priorizar las observaciones por severidad y agregar pruebas de regresion si alguna representa riesgo funcional.
Maintainability	Code smells y deuda tecnica estimada.	Maintainability Rating A; 52 maintainability issues.	La calificacion global A permite defender mantenibilidad saludable. Como mejora, se deben priorizar refactorizaciones de funciones complejas, deuda tecnica y code smells de mayor impacto.
Open Issues	Issues abiertos del analisis estatico general.	52.	No invalida por si solo el Quality Gate, pero permite priorizar deuda tecnica; se deben revisar por severidad, costo y relacion con mantenibilidad.

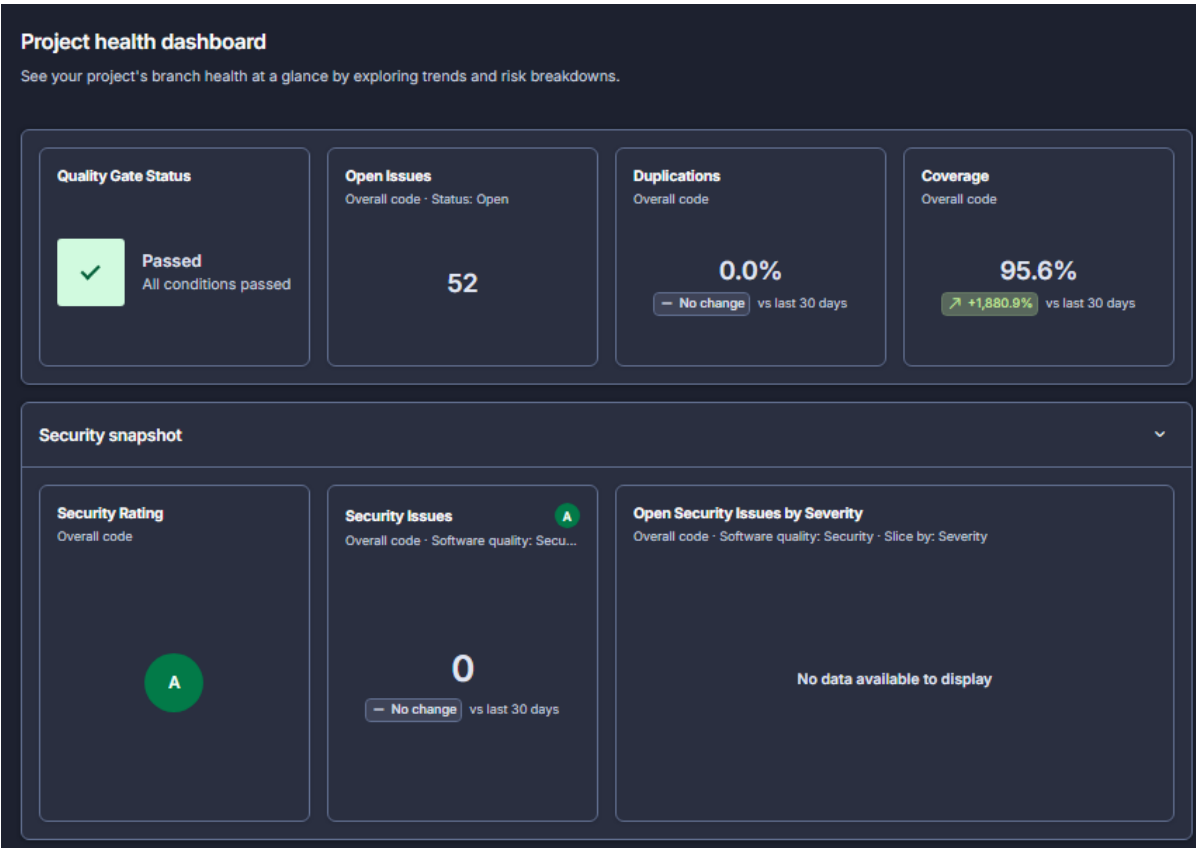
Tabla 16: Metricas SonarCloud actualizadas con evidencia complementaria de la rama `main`.

Elemento Sonar	Uso en la defensa	Evidencia
Quality Gate	Demuestra que el analisis no quedo solo configurado, sino aprobado en el flujo de PR.	Captura de Quality Gate y checks del PR final.
Coverage en main	Sustenta el criterio de pruebas y la importacion del reporte LCOV del frontend.	Captura de coverage en main.
Duplicidad y seguridad	Sustentan dos metricas Sonar adicionales a coverage: duplicidad 0.0 %, Security Rating A y Security Issues 0.	Captura general de SonarCloud en main.
Mantenibilidad y reliability	Se evidencian en la captura complementaria de SonarCloud en main: Reliability Rating A con 19 issues bajos y Maintainability Rating A con 52 issues.	Captura evidencias/38_sonarcloud_measures_main.png.

Tabla 17: Lectura de evidencia SonarCloud tomada de la rama main.



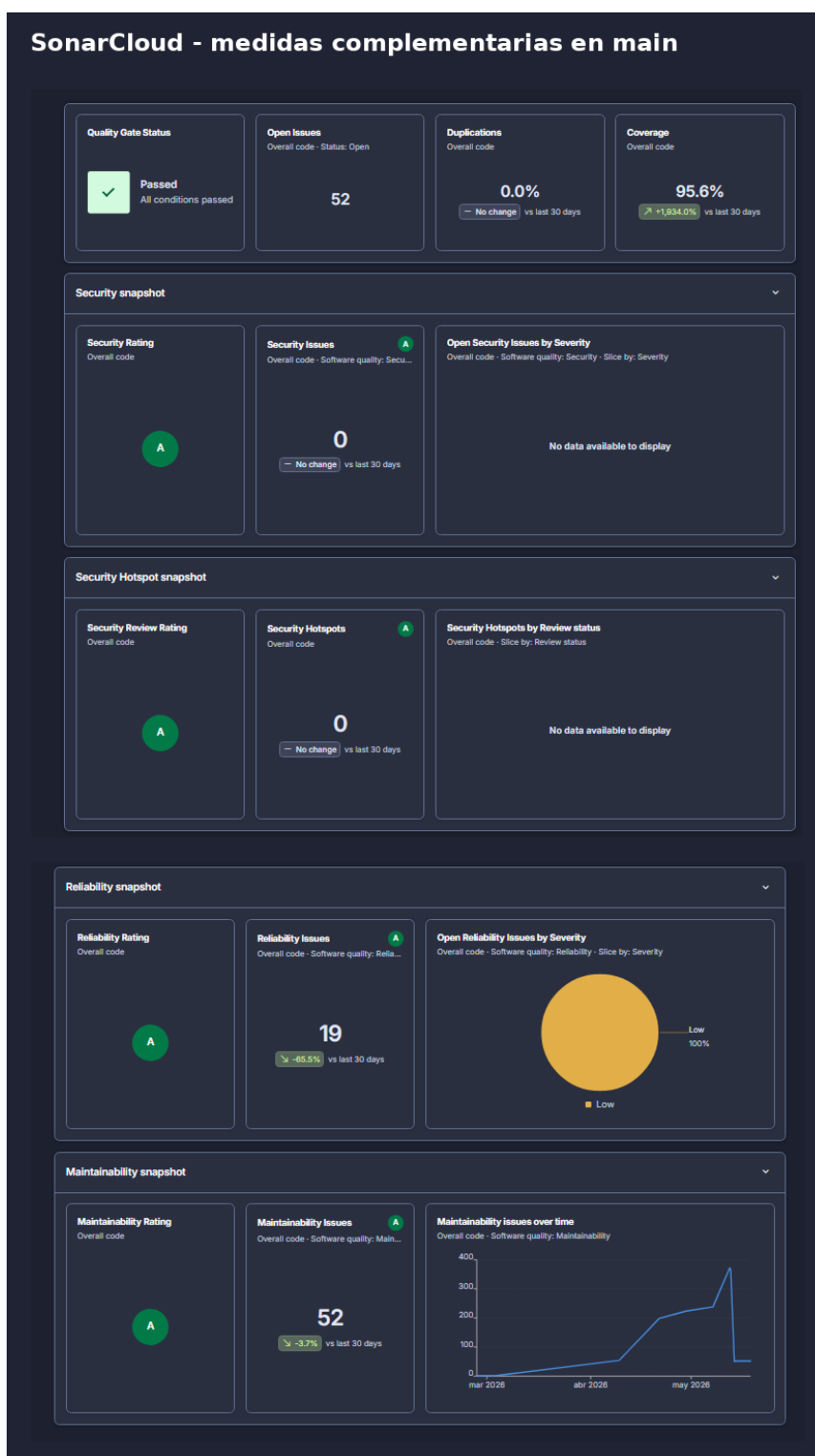
Evidencia de coverage actualizado en SonarCloud para la rama main.



Evidencia de metricas generales de SonarCloud en *main*.

5.1. Evidencia complementaria de SonarCloud

Para fortalecer la defensa del criterio de métricas de calidad y pruebas unitarias, se anexó una captura complementaria de SonarCloud en la rama *main*. Esta vista permite observar métricas adicionales a las ya mostradas en el resumen general del proyecto, especialmente seguridad, confiabilidad y mantenibilidad.



Evidencia complementaria de SonarCloud en main: Quality Gate, coverage, duplicidad, seguridad, reliability y maintainability.

La captura evidencia que el proyecto mantiene **Quality Gate aprobado**, **coverage de 95.6 %**, **duplicidad de 0.0 %**, **Security Rating A**, **Security Issues 0**, **Security Hotspots 0** y **Security Review Rating A**. Estos resultados fortalecen el cumplimiento del criterio de métricas de calidad porque demuestran que, además de las métricas propias del sistema,

el proyecto cuenta con métricas externas de análisis estático sobre cobertura, duplicidad y seguridad.

En cuanto a confiabilidad, la vista muestra **Reliability Rating A** y 19 reliability issues clasificados como bajos. Esto indica que existen observaciones técnicas detectadas por SonarCloud, pero su severidad no bloquea el Quality Gate. La acción recomendada es priorizar estas observaciones en futuras iteraciones, revisar su causa y agregar pruebas de regresión si alguna de ellas corresponde a un comportamiento riesgoso.

En cuanto a mantenibilidad, la vista muestra **Maintainability Rating A** y 52 maintainability issues. Estos hallazgos representan oportunidades de mejora asociadas a deuda técnica, legibilidad, duplicación potencial de lógica o complejidad local. No invalidan el cumplimiento de la entrega porque el rating global permanece en A y el Quality Gate está aprobado, pero sí sirven como insumo para el postmortem y para planear refactorizaciones futuras.

La interpretación final es que SonarCloud complementa las métricas propias del sistema: las métricas de DriveControl evalúan riesgo documental, completitud operativa y criticidad de alertas, mientras que SonarCloud evalúa propiedades internas del código como cobertura, duplicidad, seguridad, confiabilidad y mantenibilidad.

6. Pruebas unitarias y coverage

6.1. Alcance de pruebas

La rubrica exige coverage superior a 80 % evidenciado en SonarCloud y pruebas unitarias sobre componentes criticos. El proyecto cubre dos frentes: frontend con Vitest y reporte LCOV, y backend con `node -test` para el servicio SMS/Twilio. SonarCloud importa el LCOV del frontend desde `apps/web/coverage/lcov.info`; el backend se defiende con tests Node y evidencia de pipeline, porque no esta incluido en `sonar.sources`.

Componente critico	Riesgo cubierto	Prueba/fuente
Metricas propias	Calculo incorrecto de riesgo, completitud o criticidad.	<code>qualityMetrics.test.js</code>
Fechas y vencimientos	Clasificacion equivocada de documentos vencidos o proximos.	<code>dateUtils.test.js</code>
Formatos colombianos	Placas, telefonos o documentos con formato invalido.	<code>colombiaFormats.test.js</code>
Adaptadores de alertas	Alertas SOAT, RTM, licencia y vehiculo mal priorizadas.	<code>*AlertAdapter.test.js</code>
Simulador RUNT	Flujo de consulta o respuesta simulada inconsistente.	<code>useRUNTSimulator.test.js</code>
SMS/Twilio backend	Payload, errores, modo mock y logs con datos sensibles.	<code>backend/test/smsService.test.js</code>

Tabla 18: Componentes criticos cubiertos por pruebas unitarias.

6.2. Pruebas frontend

El frontend usa Vitest y cubre utilidades, formatos colombianos, validaciones de correo, adaptadores de alertas, reglas de fecha, metricas de calidad y simulacion RUNT. La validacion local documentada reporta 10 archivos de prueba, 126 tests exitosos y coverage local superior a 90 % en los indicadores principales.

```
npm --prefix apps/web ci
npm --prefix apps/web run lint
npm --prefix apps/web test
npm --prefix apps/web run build
```

```
● sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ npm --prefix apps/web test

> web@0.0.0 test
> sh -c 'vitest run --coverage.enabled=true "$@" --coverage.reporter=text --coverage.reporter=lcovonly --coverage.reporter=html' --

RUN v4.1.5 /home/sebas/proyectos/FIS_2610_3517_G4/apps/web
Coverage enabled with v8

✓ src/__tests__/rtmAlertAdapter.test.js (4 tests) 10ms
✓ src/__tests__/vehicleAlertAdapter.test.js (4 tests) 11ms
✓ src/__tests__/colombiaFormats.test.js (24 tests) 15ms
✓ src/__tests__/baseAlertAdapter.test.js (8 tests) 10ms
✓ src/__tests__/conductorAlertAdapter.test.js (4 tests) 12ms
✓ src/__tests__/dateUtils.test.js (11 tests) 11ms
✓ src/__tests__/emailValidation.test.js (6 tests) 9ms
✓ src/test/useRUNTSimulator.test.js (11 tests) 12ms
✓ src/__tests__/soatAlertAdapter.test.js (5 tests) 14ms
✓ src/__tests__/qualityMetrics.test.js (49 tests) 29ms

Test Files 10 passed (10)
Tests 126 passed (126)
Start at 12:01:45
Duration 532ms (transform 668ms, setup 0ms, import 1.09s, tests 133ms, environment 2ms)

% Coverage report from v8
-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files    | 98.01   | 93.31   | 100     | 98.82   |
hooks      | 100     | 100     | 100     | 100     |
  useRUNTSimulator.js | 100     | 100     | 100     | 100     |
patterns/adapters | 100     | 92.45   | 100     | 100     |
  BaseAlertAdapter.js | 100     | 100     | 100     | 100     |
  ConductorAlertAdapter.js | 100     | 100     | 100     | 100     |
  RtmAlertAdapter.js | 100     | 92.85   | 100     | 100     | 5
  SoatAlertAdapter.js | 100     | 80      | 100     | 100     | 5
  VehicleAlertAdapter.js | 100     | 100     | 100     | 100     |
utils      | 97.66   | 93.1    | 100     | 98.59   |
  colombiaFormats.js | 100     | 100     | 100     | 100     |
  dateUtils.js       | 89.55   | 73.58   | 100     | 93.22   | 21,48,56,118
  emailValidation.js | 96.29   | 96.15   | 100     | 100     | 17
  qualityMetrics.js  | 100     | 99.13   | 100     | 100     | 147
-----|-----|-----|-----|-----|-----
● sebas@TinTin:~/proyectos/FIS_2610_3517_G4$
```

Ejecucion exitosa de pruebas frontend.

```
✓ built in 3.71s
● sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ npm --prefix apps/web run build

> web@0.0.0 build
> node tools/generate-llms.js && vite build --outDir ../../dist/apps/web --emptyOutDir

[generate-llms] No hay generacion de llms configurada; se continua con el build de Vite.
vite v7.3.2 building client environment for production...
✓ 1712 modules transformed.
../../dist/apps/web/index.html      0.61 kB | gzip: 0.39 kB
../../dist/apps/web/assets/index-DWiz5DWS.css 46.60 kB | gzip: 8.45 kB
../../dist/apps/web/assets/index-rcrsPGCn.js 491.77 kB | gzip: 137.21 kB
✓ built in 3.71s
```

Build frontend exitoso.

6.3. Pruebas backend

El backend incluye pruebas sobre `backend/services/smsService.js`. Estas validan contrato tecnico con Twilio, envio de OTP, errores por variables faltantes, modo mock y logs sin imprimir credenciales reales. La evidencia versionada indica 10 pruebas SMS exitosas.

```
npm --prefix backend ci
npm --prefix backend test
```

```
● sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ npm --prefix backend test

> backend@1.0.0 test
> node --test

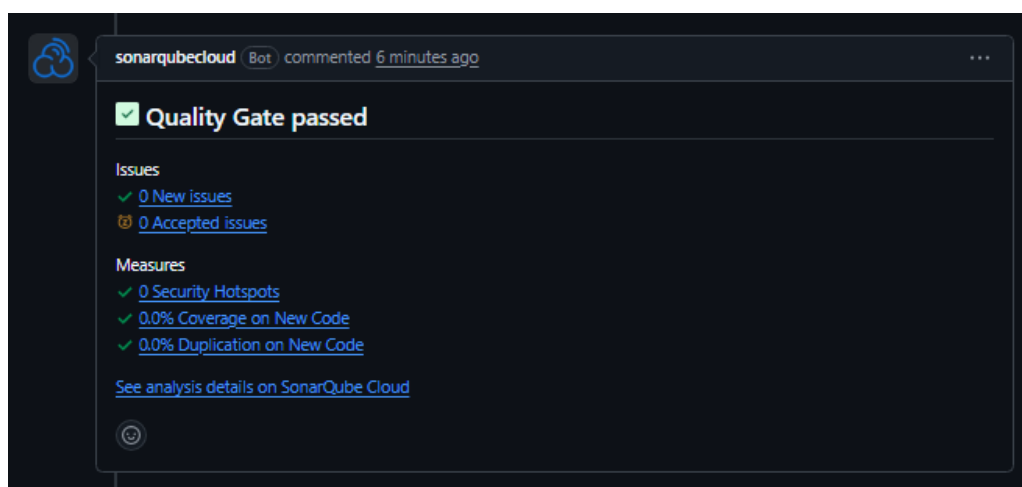
✓ envia codigo de verificacion por Twilio cuando existe sid (2.676158ms)
✓ envia codigo de recuperacion por Twilio cuando existe sid (1.74439ms)
✓ lanza error controlado cuando falta TWILIO_ACCOUNT_SID (0.786167ms)
✓ lanza error controlado cuando falta TWILIO_AUTH_TOKEN (0.416717ms)
✓ lanza error controlado cuando falta TWILIO_PHONE_NUMBER (0.555306ms)
✓ lanza error cuando Twilio no retorna sid (0.456452ms)
✓ propaga excepcion cuando Twilio responde con error (0.50744ms)
✓ no imprime credenciales Twilio en logs de envio exitoso (0.551624ms)
i tests 8
i suites 0
i pass 8
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration ms 197.71612
```

Ejecucion exitosa de pruebas backend.

6.4. Coverage local y SonarCloud

El flujo esperado es: generar LCOV local con Vitest, subirlo en GitHub Actions y permitir que SonarCloud lo importe desde `apps/web/coverage/lcov.info`. Coverage superior a 80 % significa que la mayor parte del código medido fue ejecutada por pruebas automatizadas; no garantiza ausencia de defectos, pero reduce el riesgo de regresiones silenciosas en reglas críticas. Si el coverage baja, la acción técnica es abrir el reporte HTML, priorizar archivos con menor cobertura, agregar pruebas con aserciones reales y repetir el análisis antes del merge.

Por limitación del plan actual, SonarCloud analiza oficialmente la rama `main`. Por eso el cierre se sustenta con el flujo `feature-sarm-m ->develop ->main` y con capturas posteriores al merge. Las ramas secundarias pueden tener visibilidad limitada en SonarCloud y no deben presentarse como evidencia oficial del criterio.



Quality Gate aprobado en PR de Sprint 13.

7. Docker, despliegue y red

7.1. Arquitectura de contenedores

El proyecto define tres servicios principales en Docker Compose:

Servicio	Acceso local	Responsabilidad y build
frontend	localhost:3000	Aplicación React/Vite servida por Nginx y proxy /api; build desde <code>apps/web/Dockerfile</code> .
backend	localhost:5000	API Node/Express, MongoDB, OTP y SMS; build desde el <code>Dockerfile</code> raíz.
mongodb	localhost:27017	Base de datos local para Compose mediante imagen <code>mongo:7</code> .

Tabla 19: Servicios del despliegue local.

El profesor puede clonar el repositorio y ejecutar:

```
make build
make up
make ps
```

Por decision academica, `apps/web/.env` se conserva sin cambios para facilitar la validacion funcional del sistema. Este informe no muestra valores reales, no reemplaza credenciales y no elimina el archivo.

Comando	Proposito en la entrega
<code>make build</code>	Construye las imagenes de frontend y backend usando la configuracion Compose.
<code>make up</code>	Levanta MongoDB, backend y frontend en segundo plano, usando el archivo de entorno academico conservado.
<code>docker compose ps</code>	Verifica que los servicios esperados esten arriba y saludables.
<code>docker network inspect drivecontrol-net</code>	Comprueba que la red explicita exista y use la subnet 172.28.0.0/16.

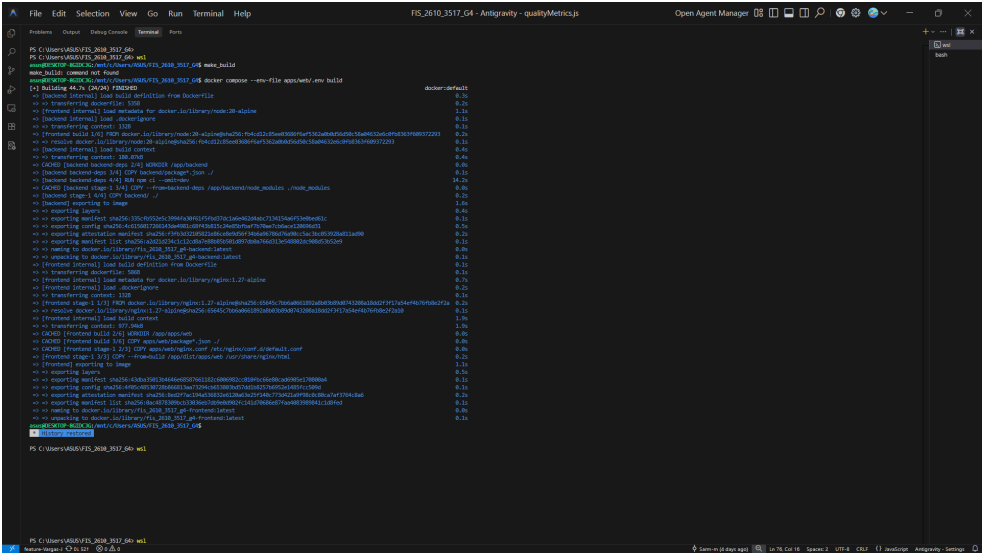
Tabla 20: Comandos de despliegue y validacion local.

Variable	Uso documentado, sin valores reales
<code>MONGO_URI / DOCKER_MONGO_URI</code>	Conexion del backend a MongoDB; en Compose se usa el servicio <code>mongodb</code> .
<code>JWT_SECRET</code>	Firma de tokens del backend.
<code>GOOGLE_CLIENT_ID / VITE_GOOGLE_CLIENT_ID</code>	Autenticacion Google en backend/frontend.
<code>EMAIL_*</code>	Configuracion de correo para OTP cuando aplica.
<code>TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER</code>	Integracion SMS con Twilio.
<code>VITE_API_URL</code>	URL o proxy de API del frontend; en Docker se usa <code>/api</code> .

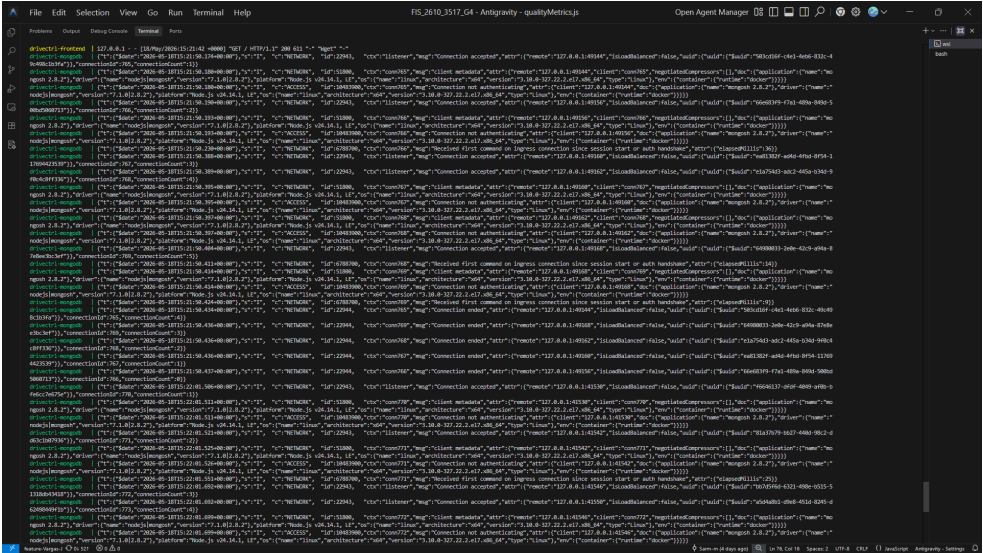
Tabla 21: Variables de entorno documentadas solo por nombre.

7.2. Red Docker

La red visible se define como `drivecontrol-net`, con driver `bridge` y subnet `172.28.0.0/16`. En `docker-compose.yml` la clave interna es `drivectl-net`, pero el nombre inspeccionable por Docker es `drivecontrol-net`. Los contenedores se comunican por nombre de servicio, no por `localhost` interno.



Salida exitosa de make build.



Salida exitosa de make up.

```

sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ docker compose config
- --spider
- -q
- http://127.0.0.1:3000/
timeout: 5s
interval: 20s
retries: 5
start_period: 10s
networks:
  drivectrl-net: null
ports:
- mode: ingress
  target: 3000
  published: "3000"
  protocol: tcp
restart: unless-stopped
mongodb:
  container_name: drivectrl-mongodb
  healthcheck:
    test:
      - CMD-SHELL
      - mongosh --quiet --eval "db.adminCommand('ping').ok" | grep 1
    timeout: 5s
    interval: 10s
    retries: 10
    start_period: 10s
  image: mongo:7
  networks:
    drivectrl-net: null
  ports:
  - mode: ingress
    target: 27017
    published: "27017"
    protocol: tcp
  restart: unless-stopped
  volumes:
  - type: volume
    source: mongo_data
    target: /data/db
    volume: {}
networks:
  drivectrl-net:
    name: drivecontrol-net
    driver: bridge
    ipam:
      config:
      - subnet: 172.28.0.0/16
volumes:
  mongo_data:
    name: fis_2610_3517_g4_mongo_data

```

Validacion de configuracion Docker Compose.

```

sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ docker compose up -d
docker compose ps
[+] Running 4/4
✓ Network drivecontrol-net      Created                                0.1s
✓ Container drivectrl-mongodb   Healthy                             8.8s
✓ Container drivectrl-backend   Healthy                             13.5s
✓ Container drivectrl-frontend   Started                             13.6s

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
drivectrl-backend	fis_2610_3517_g4-backend	"docker-entrypoint.s..."	backend	15 seconds ago	Up 7 seconds (healthy)	0.0.0.0:5000->5000/tcp
drivectrl-frontend	fis_2610_3517_g4-frontend	"/docker-entrypoint..."	frontend	15 seconds ago	Up Less than a second (health: starting)	80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
drivectrl-mongodb	mongo:7	"docker-entrypoint.s..."	mongodb	16 seconds ago	Up 14 seconds (healthy)	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp

Servicios levantados con Docker Compose.

```

sebas@TinTin:~/proyectos/FIS_2610_3517_G4$ docker network inspect drivecontrol-net
{
  "Name": "drivecontrol-net",
  "Id": "cdf990798cf1bf596d00762e20a82840f423d10cbb750fb838f29898cd43545e",
  "Created": "2026-05-15T12:05:36.065713179-05:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.28.0.0/16",
        "IPRange": "",
        "Gateway": "172.28.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Options": {},
  "Labels": {
    "com.docker.compose.config-hash": "35d5a9c0e586a7f4547c77a016d6a4f4e9bfd88405d74feee07b1075a264f5c5",
    "com.docker.compose.network": "drivectrl-net",
    "com.docker.compose.project": "fis_2610_3517_g4",
    "com.docker.compose.version": "2.40.3"
  },
  "Containers": {
    "5ce6357fc7008e91c5ec0f0ffcfdd4d8d1a7a40ba4292563b52a54c8c21d1c51": {
      "Name": "drivectrl-mongodb",
      "EndpointID": "c8f836a638d1dd222dfeddd129ec3dd164395c802ae0df1cacb114c1a23ac9eb",
      "MacAddress": "82:e5:89:a0:93:5e",
      "IPv4Address": "172.28.0.2/16",
      "IPv6Address": ""
    },
    "957d19d89b2143b1f09237e7c58ed67482ad18b0c86a31b56b18026bb045ff7b": {
      "Name": "drivectrl-backend",
      "EndpointID": "44db4f84e83e76cee5d4c61b2f4d7934ee50bfbf0adfbe88f3f496e0b6c7e953",
      "MacAddress": "62:e0:e8:02:16:57",
      "IPv4Address": "172.28.0.3/16",
      "IPv6Address": ""
    },
    "af94f30db27c451899738a29dd42febb044a30fcca7c6d7dd5c1519f6fb41840": {
      "Name": "drivectrl-frontend",
      "EndpointID": "68ba59d199ddf7d8ef2a5e6180d91a3a3d16841a2cedf8d7ce4a55377cfa8ccc",
      "MacAddress": "ea:33:4f:bc:87:e2",
      "IPv4Address": "172.28.0.4/16",
      "IPv6Address": ""
    }
  }
}

```

Inspeccion de la red drivecontrol-net.

```

asus@DESKTOP-BGIDC36:/mnt/c/Users/ASUS/FIS_2610_3517_G4$ curl -i http://localhost:5000/api/health/db
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style
safe-inline/upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Access-Control-Allow-Origin: *
Content-Type: application/json, charset=utf-8
Content-Length: 279
ETag: W/"117-0060Rf3j3Txc6kIfrepSf0WmkpQ"
Date: Thu, 14 May 2026 15:15:50 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"ok":true,"state":"1","database":"logistica_db","target":{"protocol":"mongodb","hosts":["ac-szur2n9-shard-00-01.45cqzrh.mongodb.net:27017,ac-szur2n9-shard-00-01.45cqzrh.mongodb.net:27017,ac-szur2n9-shard-00-02.45cqzrh.i
tbase":"logistica_db","source":"atlas"]}}asus@DESKTOP-BGIDC36:/mnt/c/Users/ASUS/FIS_2610_3517_G4$

```

Healthcheck del backend con base de datos disponible. Refuerza despliegue y configuracion

porque evidencia que el servicio API no solo arranca, sino que se conecta correctamente con MongoDB dentro del entorno Docker.

```
asus@DESKTOP-8G1DCJG:/mnt/c/Users/ASUS/FIS_2610_3517_G4$ curl -i http://localhost:3000/
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Thu, 14 May 2026 15:17:05 GMT
Content-Type: text/html
Content-Length: 611
Last-Modified: Thu, 14 May 2026 15:11:45 GMT
Connection: keep-alive
ETag: "6a05e631-263"
Accept-Ranges: bytes

<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="DriveControl – Gestión Inteligente de flotas (SOAT, Tecno, licencias y alertas)." />
    <title>SVNTIX TECH | DriveControl</title>
    <script type="module" crossorigin src="/assets/index-DztR6fm5.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-pIMCqarc.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>asus@DESKTOP-8G1DCJG:/mnt/c/Users/ASUS/FIS_2610_3517_G4$
```

Respuesta HTTP 200 del frontend. Soporta la defensa del despliegue al mostrar que la interfaz queda servida y accesible despues de levantar el stack.

```
</html>asus@DESKTOP-8G1DCJG:/mnt/c/Users/ASUS/FIS_2610_3517_G4$ curl -i http://localhost:3000/api/health/db
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Thu, 14 May 2026 15:18:25 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 270
Connection: keep-alive
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: data:;unsafe-inline https: data:;unsafe-request
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Access-Control-Allow-Origin: *
ETag: W/"117-0Q5Rf5J3s1xc6k1foepSf0AakpQ"

{"ok":true,"state":1,"database":"logistica_db","target":{"protocol":"mongodb","hosts":["ac-szur2n9-shard-00-00.45cqzzh.mongodb.net:27017,ac-szur2n9-shard-00-01.45cqzzh.mongodb.net:27017,ac-szur2n9-shard-00-02.45cqzzh.mongodb.net:27017","source":"atlas"]}}asus@DESKTOP-8G1DCJG:/mnt/c/Users/ASUS/FIS_2610_3517_G4$
```

Validacion del proxy del frontend hacia el endpoint de salud de la API. Demuestra networking entre servicios y configuracion correcta del proxy /api.

Si existe conflicto con contenedores o redes previas, la accion documentada es apagar el stack con `docker compose down`. Si persiste un conflicto de volumen, puerto o red, usar `docker compose down -v`, liberar los puertos 3000, 5000 y 27017, y revisar que ninguna red externa use 172.28.0.0/16 antes de volver a levantar el proyecto.

8. CI/CD

8.1. Workflows

La entrega identifica estos workflows principales:

Workflow	Activacion	Funcion principal
ci_verificacion.yml	push, PR, manual	Instala frontend, ejecuta lint y pruebas.
sonarcloud.yml	push, PR, manual	Genera coverage, metricas propias, build y analisis SonarCloud.
docker_ci_cd.yml	Cambios Docker/app, manual	Valida frontend/backend, Compose, imagenes, health-checks y publicacion condicionada.
cd_entrega.yml	main/develop	Construye frontend y deja artefacto descargable.
HU454 auth CI/CD	push, PR, manual	Valida autenticacion y hooks externos opcionales. Archivo: pipeline_hu454_auth_ci_cd.yml.

Tabla 22: Workflows principales de CI/CD.

Los jobs marcados como omitidos en algunos runs se interpretan como ejecuciones condicionadas: por ejemplo, publicacion o despliegue cuando faltan secrets, cuando el evento es PR o cuando la rama no corresponde. No deben presentarse como despliegue real si el job de publicacion no corrio en verde.

Etapas CI/CD	Que valida	Evidencia esperada
Build y pruebas	Instala dependencias, ejecuta lint, audit, pruebas frontend/backend y build Vite.	Checks verdes en PR o GitHub Actions.
SonarCloud	Genera LCOV, ejecuta metricas propias, build y scanner.	Quality Gate Passed y coverage en main.
Docker build	Construye imagen backend desde Dockerfile e imagen frontend desde apps/web/Dockerfile.	Job <code>docker-validate</code> y logs de build.
Publicacion DockerHub	Publica imagenes con tags por SHA, rama-run y rama-latest.	Capturas DockerHub tags o run <code>docker-publish</code> .
Despliegue Compose	Descarga imagenes publicadas y levanta stack con <code>docker-compose.prod.yml</code> .	Job <code>docker-deploy</code> , healthchecks y <code>docker compose ps</code> .

Tabla 23: Secuencia defendible del pipeline CI/CD.

sonarqubecloud Bot commented 6 minutes ago

✅ **Quality Gate passed**

Issues

- ✅ 0 New issues
- ✅ 0 Accepted issues

Measures

- ✅ 0 Security Hotspots
- ✅ 0.0% Coverage on New Code
- ✅ 0.0% Duplication on New Code

[See analysis details on SonarQube Cloud](#)

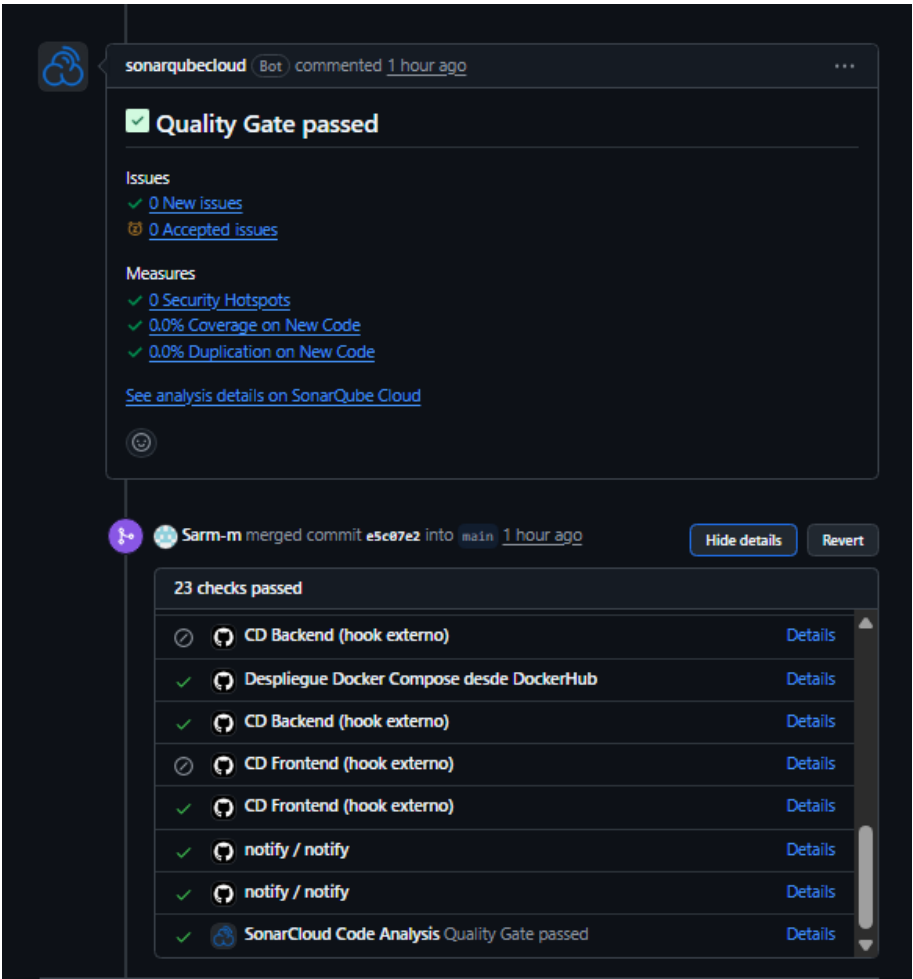
🔗 **All checks have passed**
6 skipped, 10 successful checks

- ✅ Entrega CI - Verificación del proyecto / verificación (pull_request) Successful in 15s
- ✅ Entrega CI - Verificación del proyecto / verificación (push) Successful in 13s
- ✅ HU-454 - Pipeline CI/CD Autenticación / Backend CI (instalación + validación) (pull_request) Successful in 13s
- ✅ HU-454 - Pipeline CI/CD Autenticación / Frontend CI (lint + build) (pull_request) Successful in 13s
- ✅ HU-454 - Pipeline CI/CD Autenticación / notify / notify (pull_request) Successful in 2s
- ✅ SonarCloud Analysis / SonarCloud Analysis (pull_request) Successful in 59s
- ✅ SonarCloud Analysis / SonarCloud Analysis (push) Successful in 1m
- ✅ SonarCloud Code Analysis Successful in 25s — Quality Gate passed

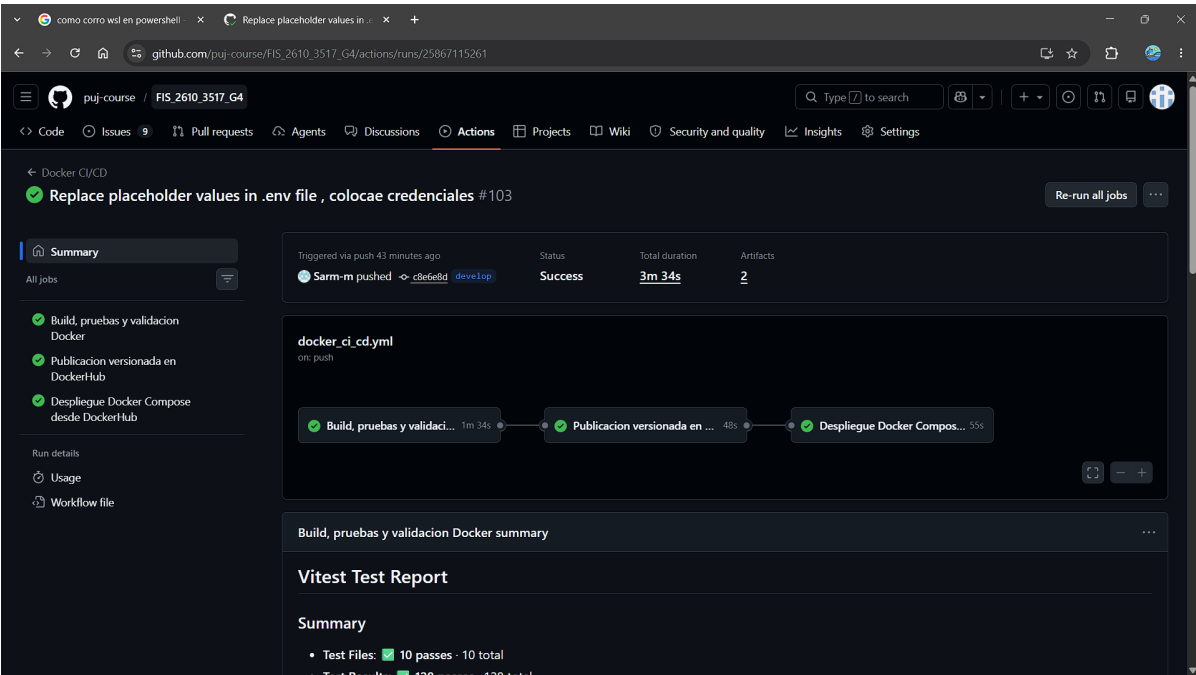
✅ **No conflicts with base branch**
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

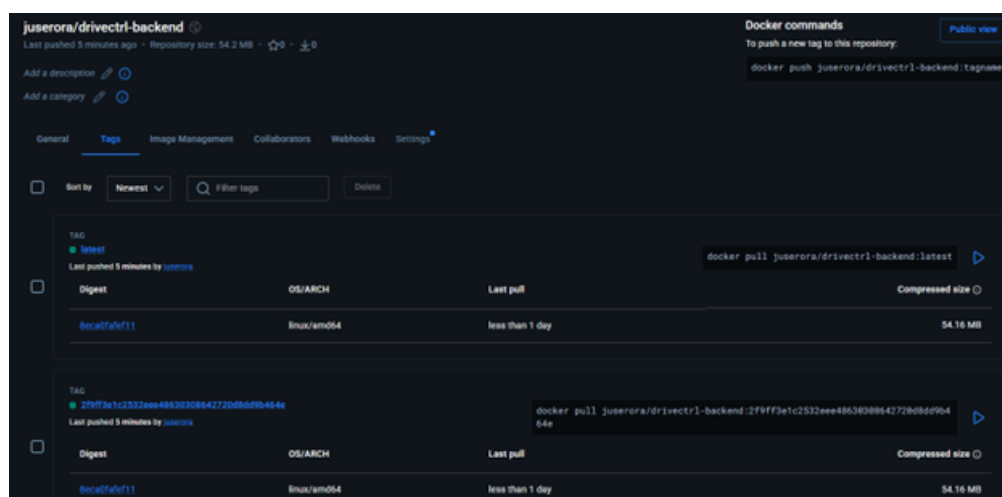
Checks aprobados en PR #623.



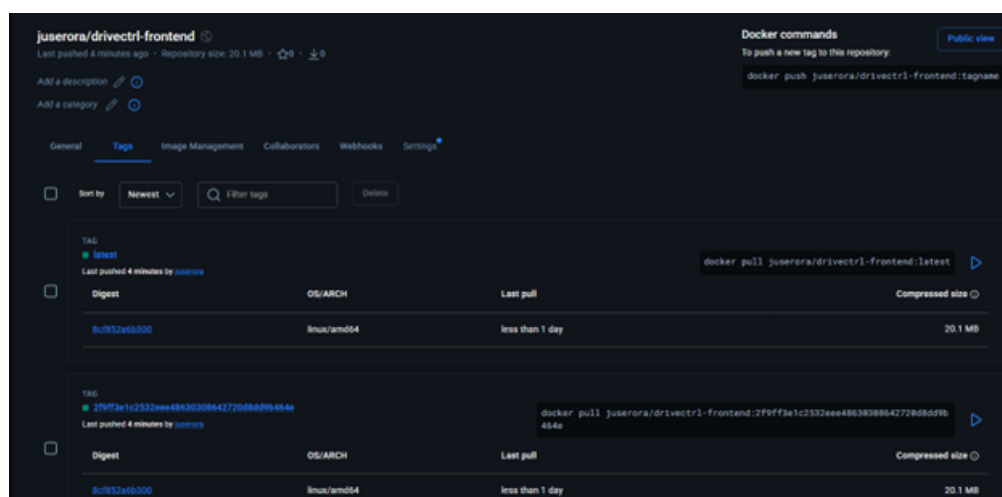
Checks del PR #625 hacia main.



Evidencia complementaria de validacion Docker en GitHub Actions.



Evidencia complementaria de tags DockerHub.



Evidencia complementaria de tags DockerHub.

9. Integracion SMS / Twilio

9.1. Problema inicial y causa

El error documentado fue: *Servicio SMS no configurado: faltan TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN o TWILIO_PHONE_NUMBER*. La causa fue que las variables necesarias se conservaban en `apps/web/.env`, mientras el backend no siempre las cargaba desde ese archivo en ejecucion local o Compose.

9.2. Solucion

El backend integra Twilio REST API en `backend/services/smsService.js`. El flujo permite OTP por SMS, maneja errores y registra informacion operativa enmascarada. Docker Compose carga `apps/web/.env` para el backend mediante `env_file`, sin imprimir valores reales.

Flujo funcional defendible:

1. El usuario solicita registro o recuperacion y selecciona verificacion por SMS.
2. El backend genera un codigo OTP con expiracion y limite de intentos.
3. El servicio SMS construye la solicitud a Twilio con autenticacion segura desde variables de entorno.
4. Twilio responde con estado de envio o error controlado.
5. El backend registra informacion operativa sin imprimir token, SID completo, telefono completo ni codigo OTP en logs publicos.

Variables requeridas, sin valores reales:

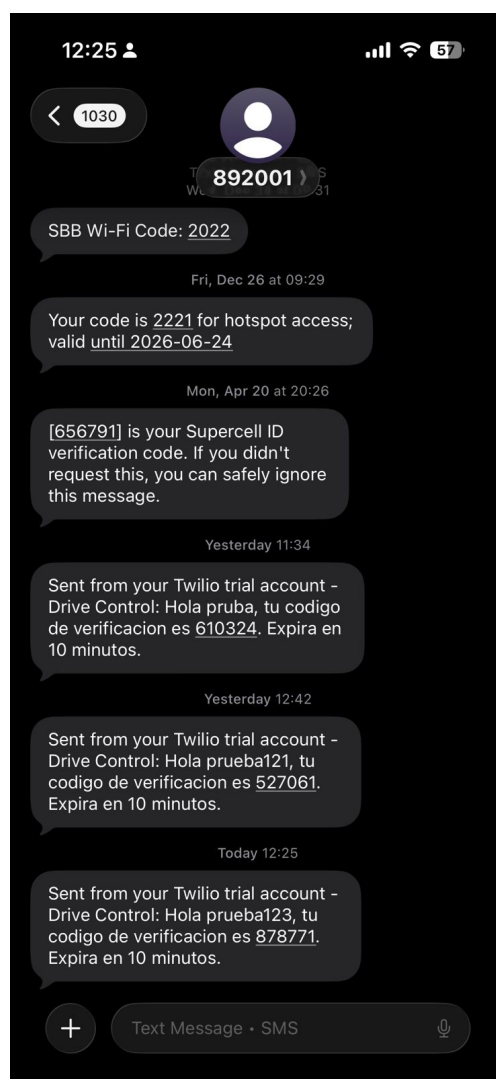
```
TWILIO_ACCOUNT_SID  
TWILIO_AUTH_TOKEN  
TWILIO_PHONE_NUMBER  
OTP_EXPIRACION_MINUTOS  
OTP_MAX_INTENTOS  
OTP_COOLDOWN_SEGUNDOS
```

Para pruebas academicas seguras tambien existe modo mock:

```
SMS_PROVIDER=mock  
SMS MOCK_ENABLED=true
```



Evidencia academica controlada: verificacion SMS desde la aplicacion.



Evidencia academica controlada: recepcion o registro exitoso en Twilio.

Estas capturas se usan como evidencia academica controlada. El informe no transcribe telefonos, codigos OTP, SID, tokens, correos ni valores de credenciales; tampoco muestra el contenido de `apps/web/.env`. Si el PDF se publica fuera del contexto de evaluacion, se recomienda generar una copia censurada de las mismas capturas. El archivo `apps/web/.env` se conserva por decision academica para validacion del profesor; en produccion real estos valores se cargarian desde GitHub Secrets, secretos del entorno o un gestor de secretos.

10. Trazabilidad tecnica

La trazabilidad tecnica conecta la HU padre, sub-issues, commits, PRs y evidencias capturadas. Los PRs #623 y #625 se usan aqui como evidencia de cierre e integracion final, no como resumen completo de la metodologia agil del semestre.

Consolidar entrega técnica final según rúbrica #618

Closed

Task

4 / 4

Sarm-m opened 40 minutes ago

Member

HU-S13-01 — Consolidar entrega técnica final según rúbrica

Descripción

Como equipo de desarrollo de DriveControl / AutoMinder Enterprise, quiero consolidar la entrega técnica final del sistema con métricas, pruebas, Docker, CI/CD, SMS y documentación de evidencia, para demostrar el cumplimiento de la rúbrica final y sustentar el proyecto con trazabilidad verificable.

Objetivo

Organizar y validar los entregables técnicos principales de la rúbrica, dejando evidencia clara en el repositorio y comandos reproducibles para la sustentación final.

Alcance

- Consolidar documentación final en docs/final-evaluation/.
- Validar métricas propias y métricas de SonarCloud.
- Evidenciar pruebas unitarias y coverage.
- Validar Docker Compose, red Docker y despliegue local.
- Documentar CI/CD y publicación en DockerHub.
- Verificar integración SMS o modo de prueba documentado.
- Preparar comandos y evidencias para la sustentación.
- Mantener trazabilidad mediante commits, PR y cierre automático.

Criterios de aceptación

- Existe documentación organizada por criterio de rúbrica en docs/final-evaluation/.
- La documentación diferencia métricas propias del sistema y métricas de SonarCloud.
- Las pruebas unitarias frontend y backend se ejecutan correctamente.
- Docker Compose queda validado con una red explícita y documentada.
- La integración SMS queda documentada con configuración, variables y evidencia esperada.
- El README final de evaluación incluye comandos de validación y rutas de evidencia.
- Los commits asociados siguen Conventional Commits y cierran las issues correspondientes.

Sub-issues

4 of 4

Task

Consolidar métricas de calidad, SonarCloud y coverage #619

Task

Validar Docker Compose, red Docker y despliegue local #620

Task

Documentar CI/CD, DockerHub y validación de workflows #621

Task

Consolidar SMS, seguridad de entorno y paquete final de evidencia #622

Assignees

Sarm-m

Labels

Despliegue y Documentación L

documentation

Type

Task

Fields

Give feedback

Priority

None yet

Effort

None yet

Projects

FIS_2610_3517_G4

Status

Done

Milestone

Milestone 4- Dashboard, alertas y cierre del sistema

Closed yesterday, 100% complete

Relationships

None yet

Development

Create a branch for this issue or link a pull request.

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

Transfer issue

Duplicate issue

Lock conversation

Pin issue

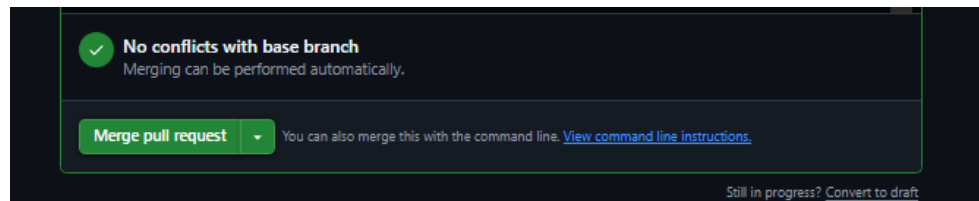
Give feedback

Historia de usuario padre #618 para la entrega tecnica final.

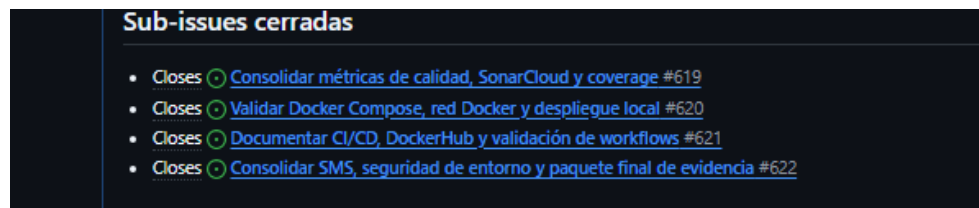
Pagina 38 de 45

Issue	Descripcion	Commit	PR	Evidencia	Estado
#618	HU padre de entrega tecnica final.	Cierre por PR mergeado y sub-issues trazadas.	#623, #625	Issue padre, PRs finales y paquete de entrega.	Cerrada
#619	Metricas, SonarCloud y coverage.	24afa71 test(coverage): consolidar pruebas y metricas de calidad. Closes #619.	#623, #625	Capturas SonarCloud, pruebas frontend y coverage.	Cerrada
#620	Docker Compose, red Docker y despliegue local.	c1fd64b chore(docker): configurar red explicita para despliegue local. Closes #620.	#623, #625	Compose config, ps y network inspect.	Cerrada
#621	CI/CD, DockerHub y workflows.	f6a9062 docs(cicd): documentar flujo de integracion y despliegue continuo. Closes #621.	#623, #625	Checks PR, Actions y tags DockerHub.	Cerrada
#622	SMS, seguridad y paquete final de evidencia.	f0e90b2 docs(evaluation): consolidar paquete final de evidencia tecnica. Closes #622. 735c1fa fix(sms): corregir configuracion de verificacion por mensajeria. Closes #622.	#623, #625	SMS app, Twilio y documentacion final.	Cerrada

El flujo final de integracion fue `feature-sarm-m ->develop ->main`: PR #623 integro la rama de cierre hacia `develop`, y PR #625 llevo `develop` a `main`.



PR #623 sin conflictos con `develop`.



Issues relacionadas en PR #623.

11. Postmortem

11.1. Que salio bien

- La base funcional permitio cerrar el sistema con frontend, backend, MongoDB, Docker y CI/CD.
- Las metricas propias quedaron implementadas, testeadas y documentadas.
- Las pruebas frontend/backend validan reglas criticas y el servicio SMS sin exponer credenciales.
- La trazabilidad final conecta issues, commits, PRs y evidencias.
- Docker Compose facilita reproducibilidad para el evaluador mediante `make build` y `make up`.

11.2. Que salio mal y causas raiz

Problema	Causa raiz tecnica	Causa raiz de proceso	Impacto	Accion correctiva	Accion preventiva	Rol responsable
Evidencia externa incompleta en fases previas.	Capturas de SonarCloud, DockerHub, Twilio y tablero no siempre quedaron versionadas al momento del cierre.	La recoleccion de evidencia se dejo parcialmente para el final de sprint.	Riesgo de defensa parcial aunque el sistema funcione.	Normalizar nombres, rutas y placeholders en el informe final.	Checklist de evidencia por HU antes de cerrar issue.	Scrum Master / QA Lead
Gestion tardia de variables de entorno.	Backend no siempre cargaba variables Twilio desde el archivo usado por Compose.	La politica de secretos y archivos de entorno no se formalizo desde el primer sprint.	Riesgo de configuracion, seguridad y retrasos de validacion.	Documentar variables solo por nombre y conservar <code>apps/web/.env</code> para validacion academica.	Usar secrets de CI/CD y gestor de secretos en despliegues reales.	Configuration Manager
Coverage y Quality Gate bajo presion.	Codigo nuevo podia entrar sin pruebas suficientes en ramas criticas.	El DoD no exigia evidencia de coverage local desde el inicio.	Riesgo de regresiones y bloqueo por SonarCloud.	Ejecutar pruebas con coverage y corregir fallos antes del merge.	Hacer coverage parte obligatoria de cada PR.	QA Lead
Datos GitHub con alias.	Git registra autores con correos y nombres diferentes.	No se normalizo identidad de autores durante el semestre.	Conteos individuales pueden ser injustos o confusos.	Presentar tabla normalizada y declarar pendientes no exportados.	Acordar identidad Git desde el Sprint 1.	Scrum Master
Backend fuera del alcance Sonar actual.	<code>sonar.sources</code> esta centrado en <code>apps/web/src</code> .	Se priorizo coverage frontend por tiempo y configuracion.	Requiere explicar que backend se valida con <code>node -test</code> .	Defender backend con pruebas Node y captura terminal.	Agregar coverage backend y configuracion Sonar ampliada.	QA Lead / DevOps
Metricas operativas criticas.	La app reporta 62 % riesgo documental, 56 % completitud y 73 % criticidad de alertas.	La calidad de datos y documentos no se resolvió completamente antes del cierre.	Riesgo operativo en cumplimiento, continuidad y priorizacion de flota.	Priorizar renovaciones, completar registros y cerrar alertas rojas.	Crear tablero semanal de saneamiento de datos y alertas.	Product Owner / QA Lead

Tabla 25: Postmortem consolidado: causas raiz, impacto, acciones y responsables.

12. Conclusiones

La entrega final centraliza la evidencia principal de la rubrica y permite defender el sistema desde tres frentes: funcionamiento tecnico, calidad verificable y proceso agil. DriveControl / AutoMinder Enterprise cuenta con frontend, backend, Docker Compose, MongoDB, pruebas, SonarCloud, CI/CD y SMS/Twilio documentados.

Las principales limitaciones no estan en la existencia del codigo, sino en la completitud y manejo responsable de evidencia externa. Las metricas propias atacan calidad de negocio que SonarCloud no mide; SonarCloud complementa con coverage, duplicidad y seguridad; Docker/CI/CD permiten reproducibilidad; y Scrum se sustenta con datos de todo el semestre. El equipo decidio no presentar tag o release final; el control de versiones se sustenta con PRs, issues, commits y merges trazables. Las capturas SMS/Twilio se usan como evidencia academica controlada y no se transcriben valores sensibles.

13. Referencias

Referencias

- [1] Repositorio GitHub: https://github.com/puj-course/FIS_2610_3517_G4
- [2] Pull Request #623: https://github.com/puj-course/FIS_2610_3517_G4/pull/623
- [3] Pull Request #625: https://github.com/puj-course/FIS_2610_3517_G4/pull/625
- [4] Issue #618: https://github.com/puj-course/FIS_2610_3517_G4/issues/618
- [5] Docker Compose: <https://docs.docker.com/compose/>
- [6] Twilio SMS API: <https://www.twilio.com/docs/sms>
- [7] Scrum Guide: <https://scrumguides.org/>
- [8] SonarCloud: <https://docs.sonarsource.com/sonarcloud/>

14. Anexos

14.1. Evidencias usadas

Categoria	Archivos usados
SonarCloud	Capturas centralizadas de coverage y metricas generales en evidencias/03_sonarcloud_coverage_main.png, evidencias/04_sonarcloud_metricas_generales.png y evidencias/38_sonarcloud_measures_main.png.
PRs y CI	Capturas de Quality Gate, checks de PR y validacion de GitHub Actions.
Pruebas	Capturas de tests frontend, build frontend y tests backend.
Docker	Capturas de Compose config, make build, make up, Compose ps, network inspect, DockerHub y Actions.
SMS/Twilio	Capturas academicas controladas en evidencias/18_sms_app_verificacion.png y evidencias/19_sms_twilio_recibido.png; no transcribir telefono, OTP, SID, token o correo.
Agile/Scrum	Capturas evidencias/25_agile_milestones_semestre.png y evidencias/27_github_contributors_semestre.png.
Arquitectura	Diagrama de base de datos centralizado en evidencias/32_diagrama_base_datos.png.
Metricas agiles finas	Tablas calculadas desde anexos/issues.json, anexos/pull_requests.json, anexos/commits_por_autor.txt y anexos/lineas_por_autor_resumen.tsv. Las reviews por integrante quedan pendientes porque requieren export adicional por API o GraphQL.
SonarCloud complementario	Captura real evidencias/38_sonarcloud_measures_main.png con vista Summary/Measures de main: Quality Gate aprobado, coverage 95.6 %, duplicidad 0.0 %, Security Rating A, Security Issues 0, Security Hotspots 0, Reliability Rating A y Maintainability Rating A.

Tabla 26: Resumen de evidencias anexadas al informe.

14.2. Evidencia complementaria de arquitectura

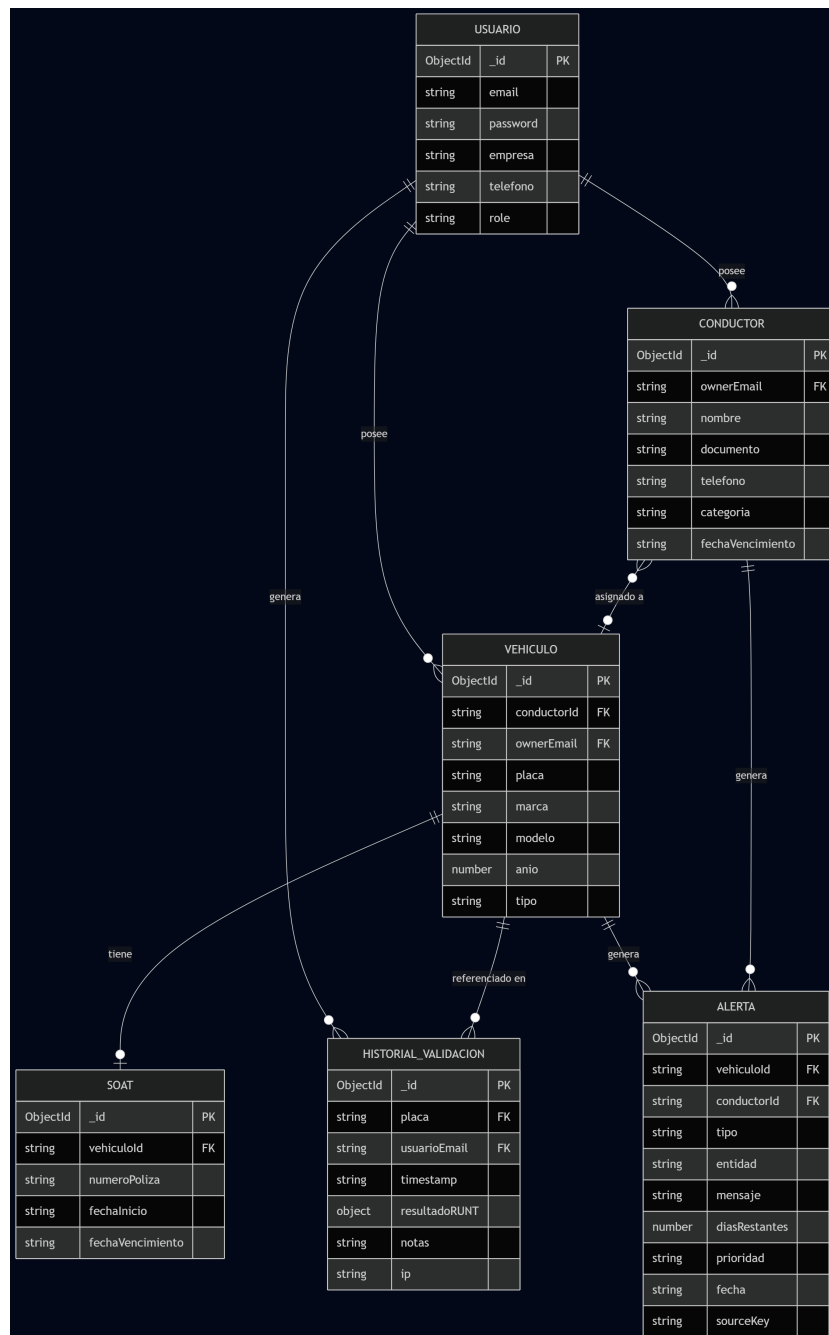


Diagrama de base de datos del sistema. No reemplaza los criterios de la rubrica, pero ayuda a explicar la estructura de datos que alimenta vehiculos, conductores, documentos, alertas y metricas propias.

14.3. Comandos de validacion

```
npm --prefix apps/web test
npm --prefix backend test
make build
```

```
make up
docker compose ps
docker network inspect drivecontrol-net
```

14.4. Commit sugerido

```
docs(report): limpiar entrega final y reforzar evidencia de rubrica.
Closes #618.
```